

kNN, наивный Байес и решающие деревья

Нелинейные модели классификации и регрессии

Елена Кантонистова
ekantonistova@hse.ru

НИУ ВШЭ



План лекции

- 1 **Метод k ближайших соседей** — идея, метрики, выбор k
- 2 **Наивный байесовский классификатор** — теорема Байеса и разбор примера
- 3 **Решающее дерево** — определение и жадный алгоритм построения
- 4 **Критерии информативности** — дисперсия, Джини, энтропия
- 5 **Ограничение сложности** — критерии останова и стрижка дерева
- 6 **Пропуски и категориальные признаки** — как их обрабатывает дерево
- 7 **Плюсы и минусы деревьев** — когда их стоит применять



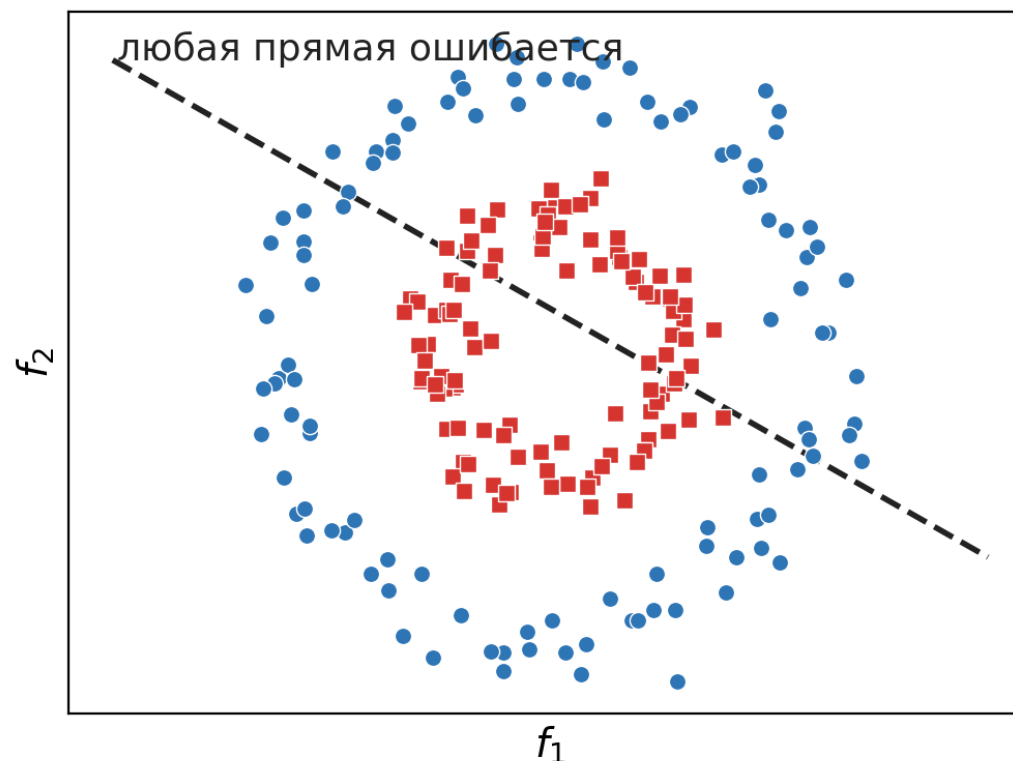
Метод k ближайших соседей

Когда линейной модели не хватает

Линейная модель проводит одну прямую (в общем случае — гиперплоскость). Если классы так не разделяются, она бессильна: как бы мы ни поворачивали прямую, ошибок останется много.

Что с этим делать

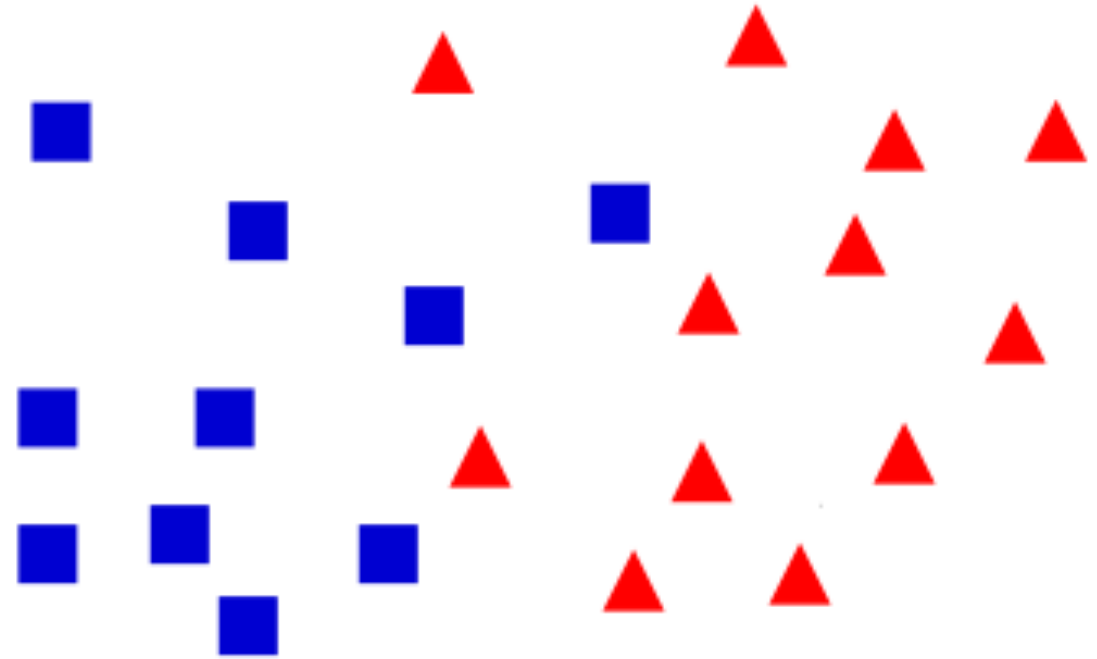
- Усложнить признаки — добавить квадраты, произведения, полиномы — и разделять уже в новом пространстве
- Взять нелинейную модель, которая сама подстраивает форму границы под данные



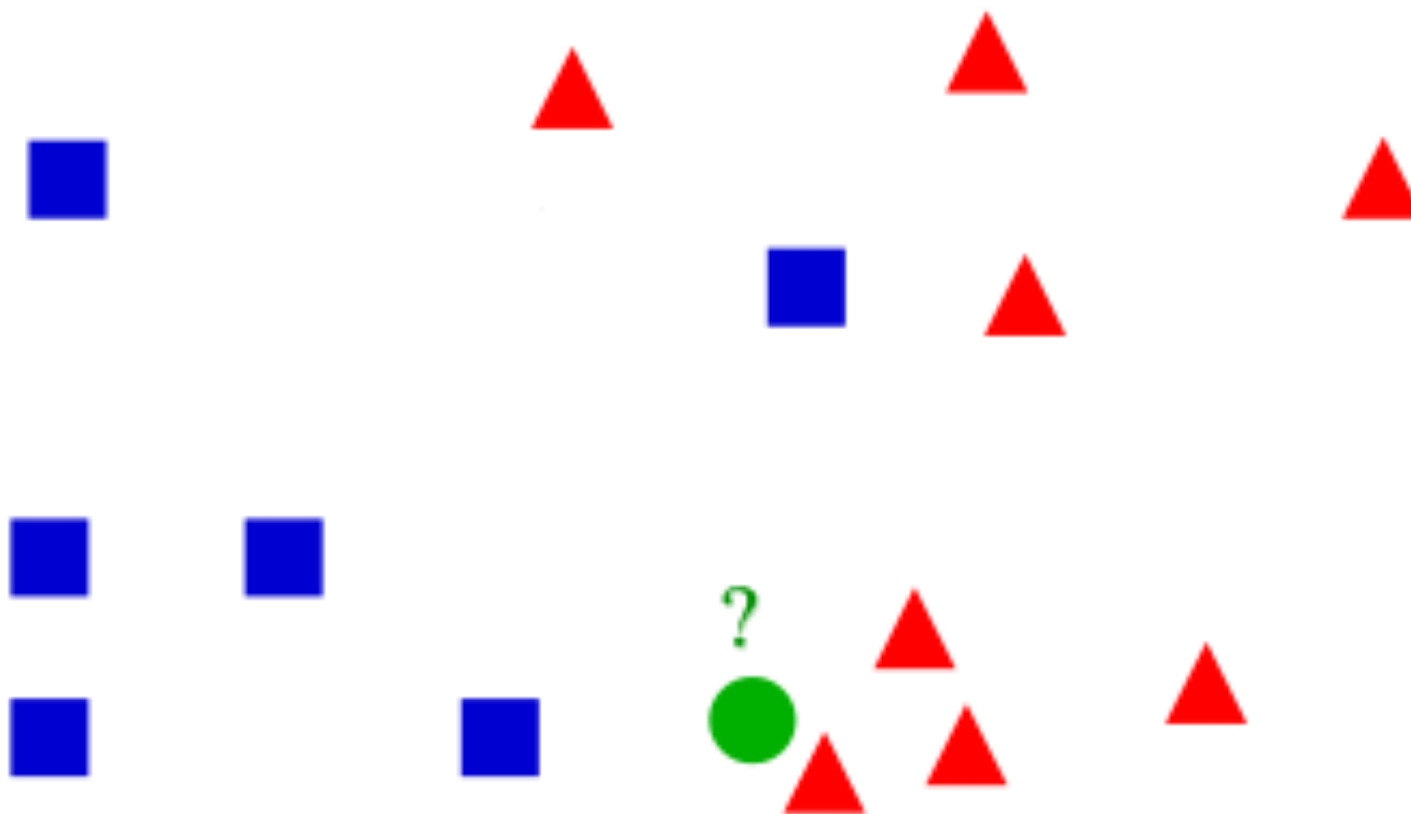
Идея метода

Схожие объекты находятся близко друг к другу в пространстве признаков

- Мы обучаем модель в классическом смысле, то есть не подбираем веса
- Вместо этого просто запоминаем всю обучающую выборку — и отвечаем на вопрос «На кого новый объект похож?»



Как классифицировать новый объект?



Зелёный объект — новый. Класа у него нет: его нужно предсказать по соседям.

Алгоритм в трёх шагах

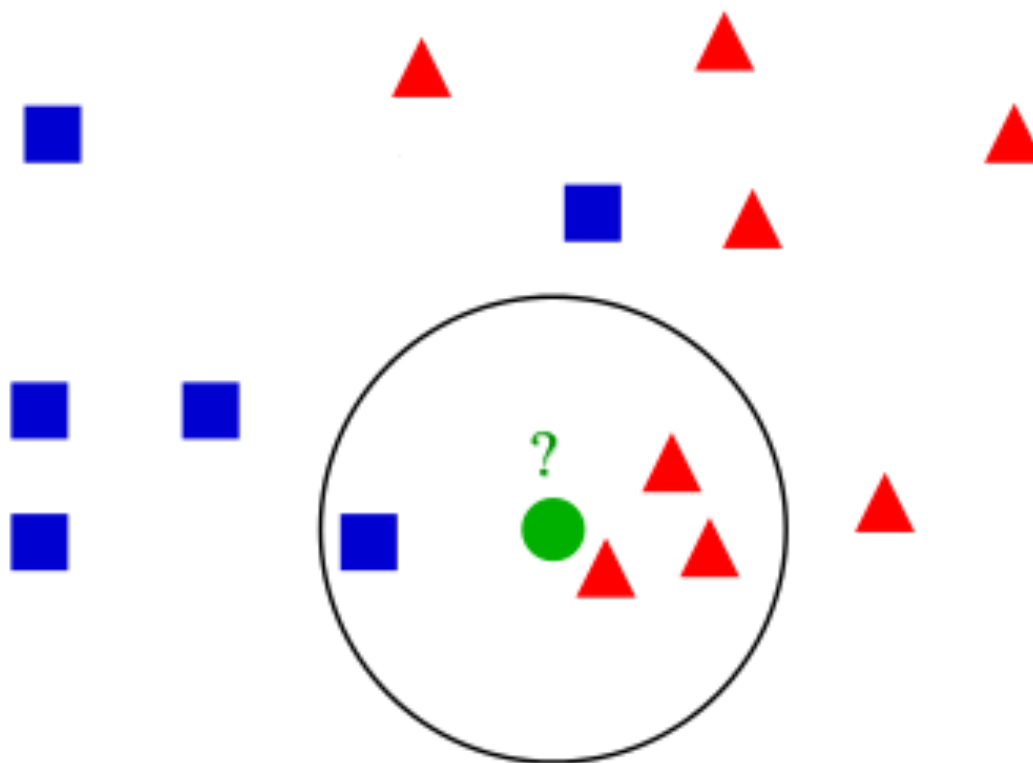
- 1 **Вычислить расстояние** — от нового объекта до каждого объекта обучающей выборки
- 2 **Выбрать k ближайших** — объектов обучающей выборки, расстояние до которых минимально
- 3 **Проголосовать** — класс нового объекта — класс, наиболее часто встречающийся среди k соседей

Обучения здесь нет

Вся начинка модели — это сама обучающая выборка. Поэтому kNN называют ленивым (lazy) алгоритмом: вся работа откладывается до момента предсказания.

Пример: $k = 4$

Число ближайших соседей k — гиперпараметр метода



Среди четырёх ближайших соседей больше треугольников — объект будет отнесён к классу треугольников.

Формализация метода

Пусть k — количество соседей. Для каждого объекта u возьмём k ближайших к нему объектов обучающей выборки:

$$x(1; u), x(2; u), \dots, x(k; u)$$

Тогда класс объекта u определяется голосованием большинства:

$$a(u) = \arg \max_{y \in Y} \sum_{i=1}^k [y(x(i; u)) = y]$$

Ближайшие объекты — это объекты, расстояние от которых до данного объекта наименьшее по некоторой метрике ρ .

Тот же метод для регрессии

Если целевая переменная вещественная — просто усредняем ответы соседей:

$$a(u) = \frac{1}{k} \sum_{i=1}^k y_{(i; u)}$$

Здесь $y_{(i; u)}$ — ответ на i -м по близости к u объекте обучающей выборки.

Взвешенный вариант

Соседей можно учитывать с весами: чем ближе объект, тем сильнее его голос. Это сглаживает границу и снижает влияние далёких соседей.

Что настраивать

Число соседей k , метрика ρ и схема весов — всё это гиперпараметры, которые подбираются по отложенной выборке.

Как измерять расстояние

Чаще всего — евклидово расстояние:

$$\rho(x, x') = \sqrt{\sum_{j=1}^d (x_j - x'_j)^2}$$

Но подойдёт любая разумная метрика:

Манхэттенское расстояние

$$\rho(x, x') = \sum_{j=1}^d |x_j - x'_j|$$

Косинусное расстояние

$$\rho(x, x') = 1 - \frac{\langle x, x' \rangle}{\|x\| \cdot \|x'\|}$$

Косинусное используют, когда важно направление вектора, а не его длина — например, для текстов. Можно задать и свою метрику под предметную область.

Обязательный шаг: масштабирование признаков

Метрика складывает все признаки в одну сумму. Если один признак измеряется в тысячах, а другой в долях единицы, первый полностью определит расстояние — а второй не будет учитываться вовсе.

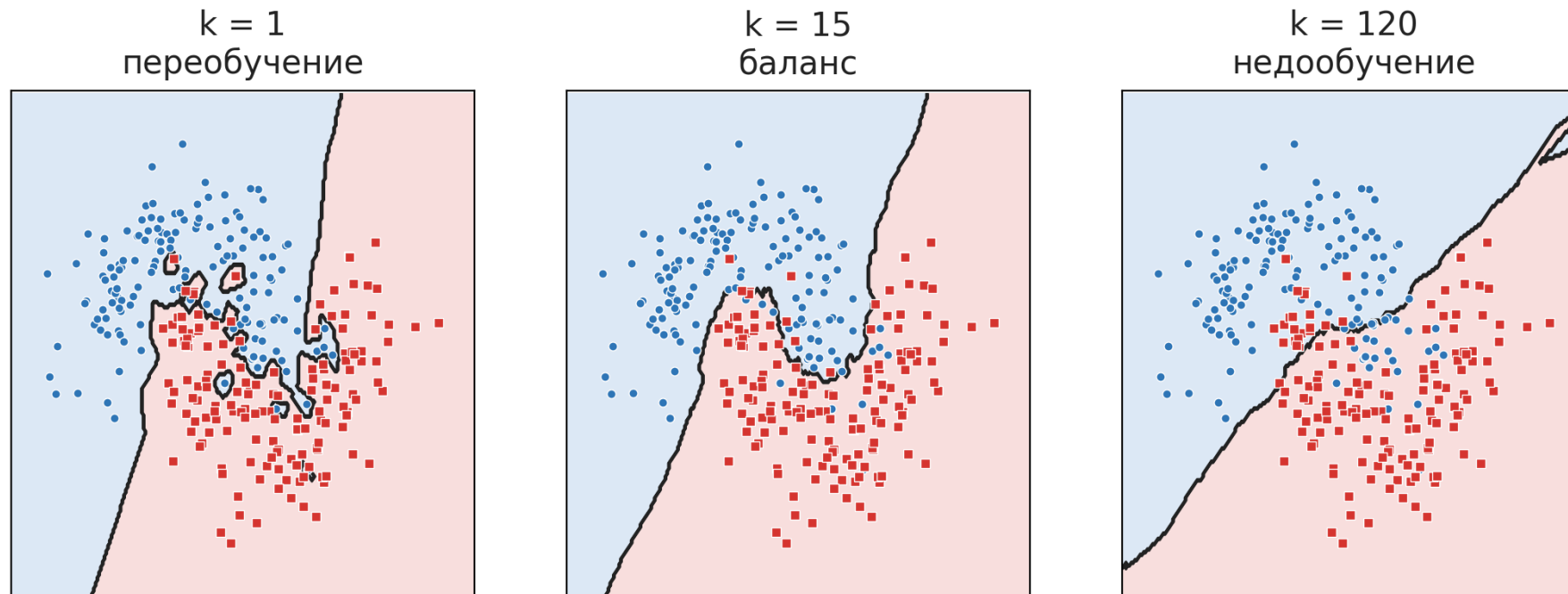
Перед использованием метода необходимо масштабировать данные, иначе признаки с большими числовыми значениями будут доминировать при вычислении расстояний!

Базовый способ — стандартизация каждого признака:

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}$$

Выбор k

k — гиперпараметр; подбирается по отложенной выборке или кросс-валидацией



Маленькое k

Граница подстраивается под каждый объект, включая шумовые, — переобучение.

Большое k

Усредняем по слишком широкой окрестности, теряем структуру данных, — недообучение.

Метод k ближайших соседей: итоги

Плюсы

- Очень простая идея, легко объяснить и реализовать
- Нет этапа обучения — “модель” готова сразу
- Естественно работает с любым числом классов
- Гибкая граница — подстраивается под данные
- Подходит и для классификации, и для регрессии

Минусы

- Предсказание медленное: нужно сравниться со всей выборкой
- Требуется хранить всю обучающую выборку в памяти
- Обязательно масштабирование признаков
- Чувствителен к шумовым объектам при малом k
- В силу отсутствия параметров может давать прогнозы низкого качества



Наивный байесовский классификатор

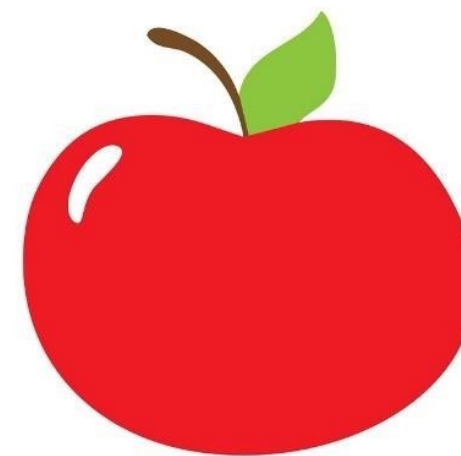
Наивный байесовский классификатор

Наивный байесовский классификатор — это алгоритм классификации, основанный на теореме Байеса с допущением о независимости признаков.

Пример: фрукт может считаться яблоком, если

- он красный
- круглый
- его диаметр составляет порядка 8 см

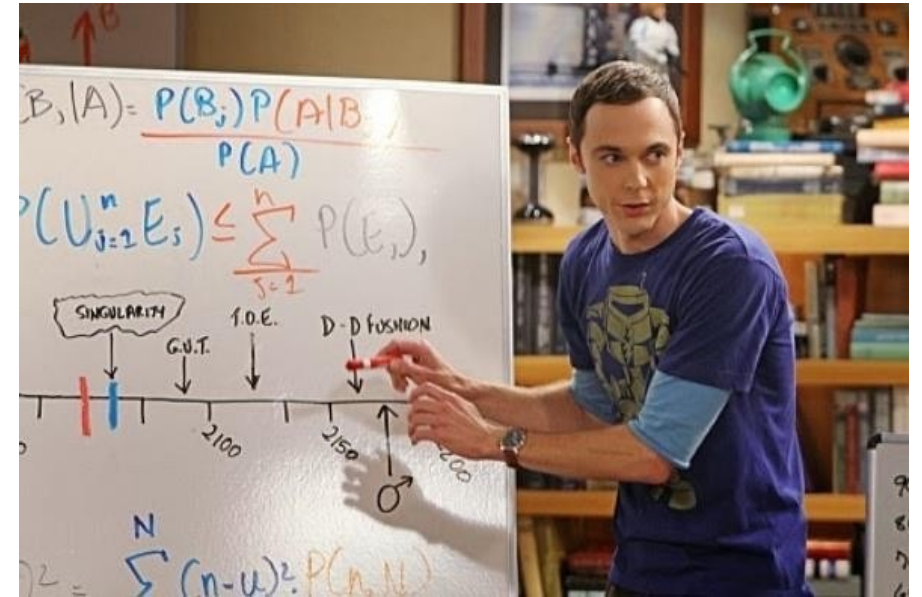
Предполагаем, что признаки вносят независимый вклад в вероятность того, что фрукт является яблоком.



Теорема Байеса

$$P(c | x) = \frac{P(x | c) \cdot P(c)}{P(x)}$$

- $P(c|x)$ — вероятность того, что объект со значением признака x принадлежит классу c
- $P(c)$ — априорная вероятность класса c
- $P(x|c)$ — вероятность того, что значение признака равно x при условии, что объект принадлежит классу c
- $P(x)$ — априорная вероятность значения признака x



Пример работы байесовского алгоритма

На основе данных о погодных условиях нужно определить, состоится ли матч

Weather	Play
Sunny	No
Overcast	Yes
Rainy	Yes
Sunny	Yes
Sunny	Yes
Overcast	Yes
Rainy	No

Weather	Play
Rainy	No
Sunny	Yes
Rainy	Yes
Sunny	No
Overcast	Yes
Overcast	Yes
Rainy	No

Данные

- 14 наблюдений
- Один категориальный признак Weather
- Бинарная целевая переменная Play

Преобразуем набор данных в таблицу частот: сколько раз каждая погода встречалась с каждым ответом.

Таблица частот

Weather	No	Yes	Всего	P(Weather)
Overcast	0	4	4	$4/14 = 0,29$
Rainy	3	2	5	$5/14 = 0,36$
Sunny	2	3	5	$5/14 = 0,36$
Grand Total	5	9	14	1,00

Априорные вероятности классов

- $P(\text{No}) = 5/14 \approx 0,36$
- $P(\text{Yes}) = 9/14 \approx 0,64$

Условные вероятности

- $P(\text{Sunny} \mid \text{Yes}) = 3/9$
- $P(\text{Sunny} \mid \text{No}) = 2/5$

Вероятности в правой части формулы Байеса вычисляются прямо по этой таблице.

Решим задачу с помощью теоремы Байеса

Вопрос: состоится ли матч, если погода солнечная?

$$P(\text{Yes} \mid \text{Sunny}) = P(\text{Sunny} \mid \text{Yes}) \cdot P(\text{Yes}) / P(\text{Sunny})$$

Все три вероятности берём из таблицы частот:

$$P(\text{Sunny} \mid \text{Yes}) = \frac{3}{9}, \quad P(\text{Sunny}) = \frac{5}{14}, \quad P(\text{Yes}) = \frac{9}{14}$$

$$P(\text{Yes} \mid \text{Sunny}) = \frac{3}{9} \cdot \frac{9}{14} : \frac{5}{14} = \frac{3}{5} = 0,6 \Rightarrow 60\%$$

В случае нескольких признаков

Пусть $x_1, \dots, x_n$ — признаки объекта, y — целевая переменная. Тогда теорема Байеса записывается в виде:

$$P(y \mid x_1, \dots, x_n) = \frac{P(x_1 \mid y) P(x_2 \mid y) \cdots P(x_n \mid y) P(y)}{P(x_1) P(x_2) \cdots P(x_n)}$$

Знаменатель одинаков для всех классов, поэтому при сравнении классов его можно отбросить — остаётся решающее правило:

$$a(x) = \arg \max_{c \in Y} P(c) \prod_{j=1}^n P(x_j \mid c)$$

Именно перемножение $P(x_j \mid c)$ по отдельным признакам и есть та самая «наивность»: мы считаем признаки независимыми внутри класса.

Почему алгоритм «наивный»

Предположение о независимости признаков внутри класса почти никогда не выполняется на реальных данных. Рост и вес человека связаны, слова в тексте связаны, погода и день недели связаны.

И тем не менее

- Оценивать n одномерных распределений гораздо проще, чем одно n -мерное
- Для решения о классе важен не точный уровень вероятности, а то, какой класс окажется больше
- На практике метод часто работает достойно даже при нарушенном предположении

Проблема нулевой вероятности

$$P(x_j = u \mid c) = 0 \quad \Rightarrow \quad P(c \mid x) = 0$$

Если в тестовых данных встретилась категория, которой не было в обучающей выборке, модель присвоит ей нулевую вероятность — и обнулит всё произведение.

Сглаживание Лапласа

Лечится добавлением единицы к каждому счётчику: тогда ни одна вероятность не обращается в ноль, а редкие категории просто получают маленький вес.

Наивный Байес на практике

`sklearn.naive_bayes`

GaussianNB

- Вещественные признаки
- Внутри каждого класса признак считается нормально распределённым

MultinomialNB

- Счётчики: частоты слов, число событий
- Классика для классификации текстов

BernoulliNB

- Бинарные признаки: слово встретилось в документе или нет

Плюсы

- Классификация быстрая и простая
- Требуется мало данных для оценки параметров
- При выполнении предположения о независимости даёт очень высокое качество

Минусы

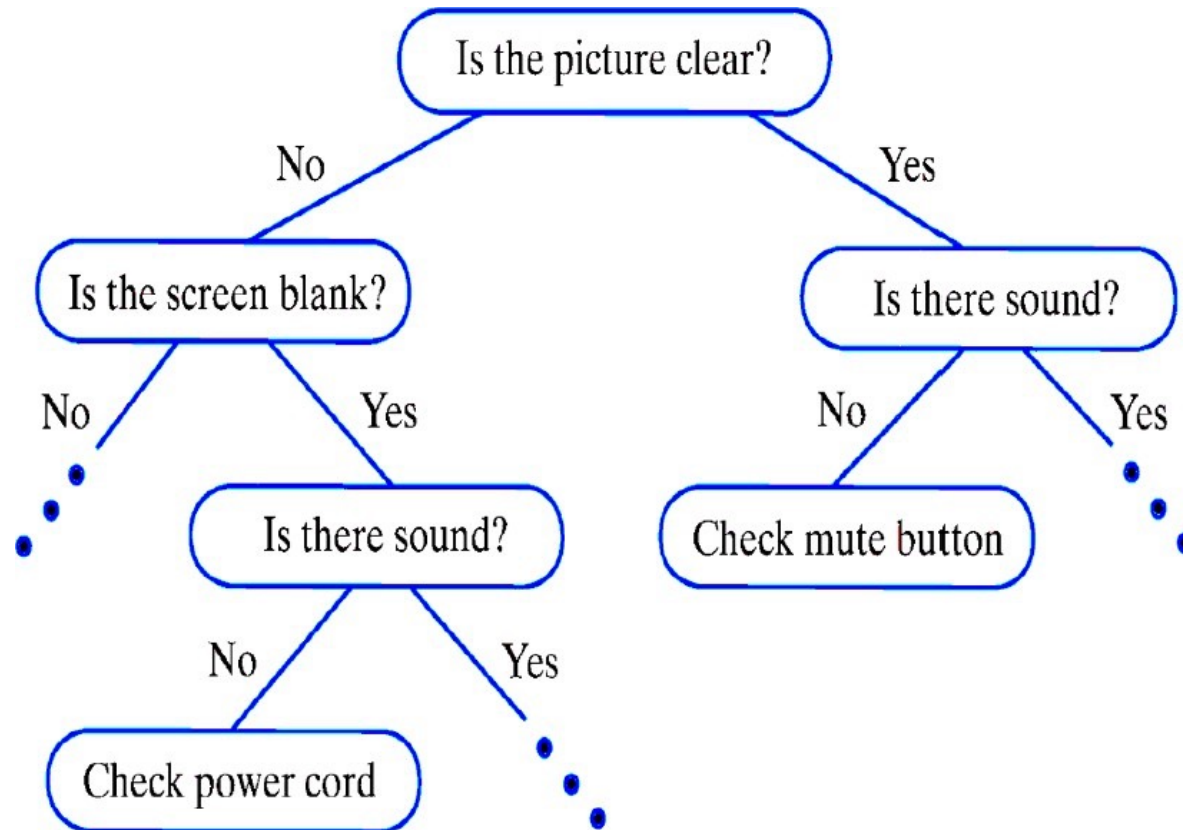
- Предположение о независимости почти всегда нарушено
- Новая категория в тесте получает нулевую вероятность
- Предсказанные вероятности плохо откалиброваны



Решающее дерево

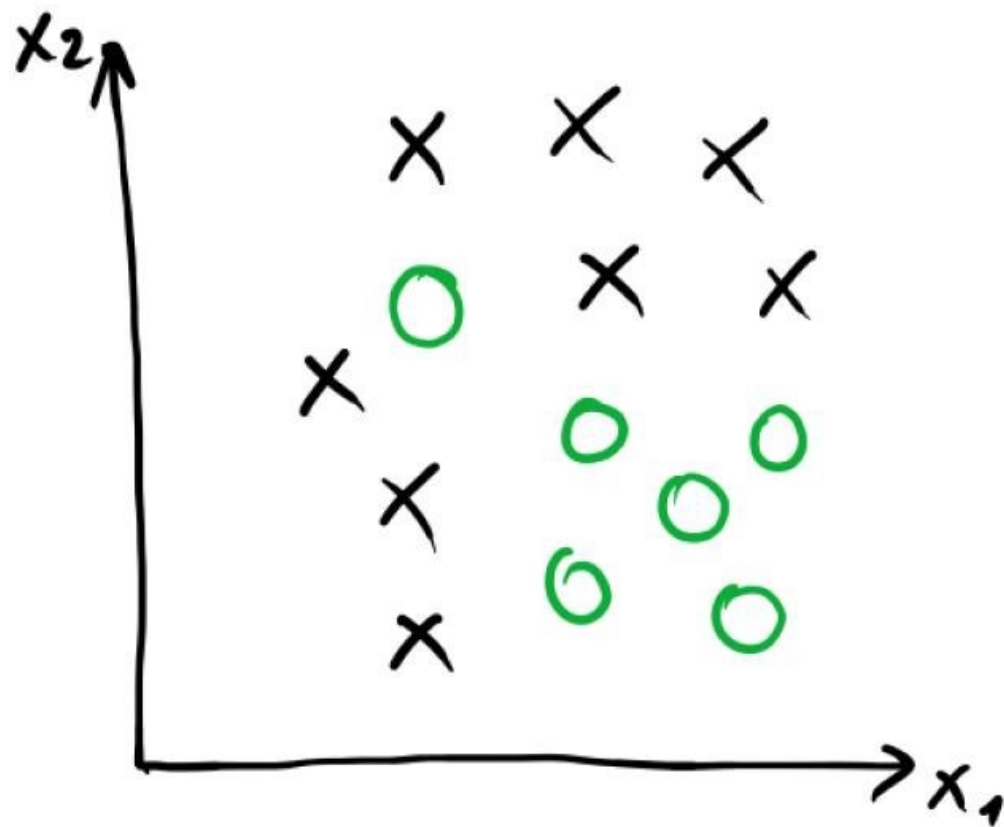
Пример решающего дерева

Бытовая диагностика: телевизор не работает

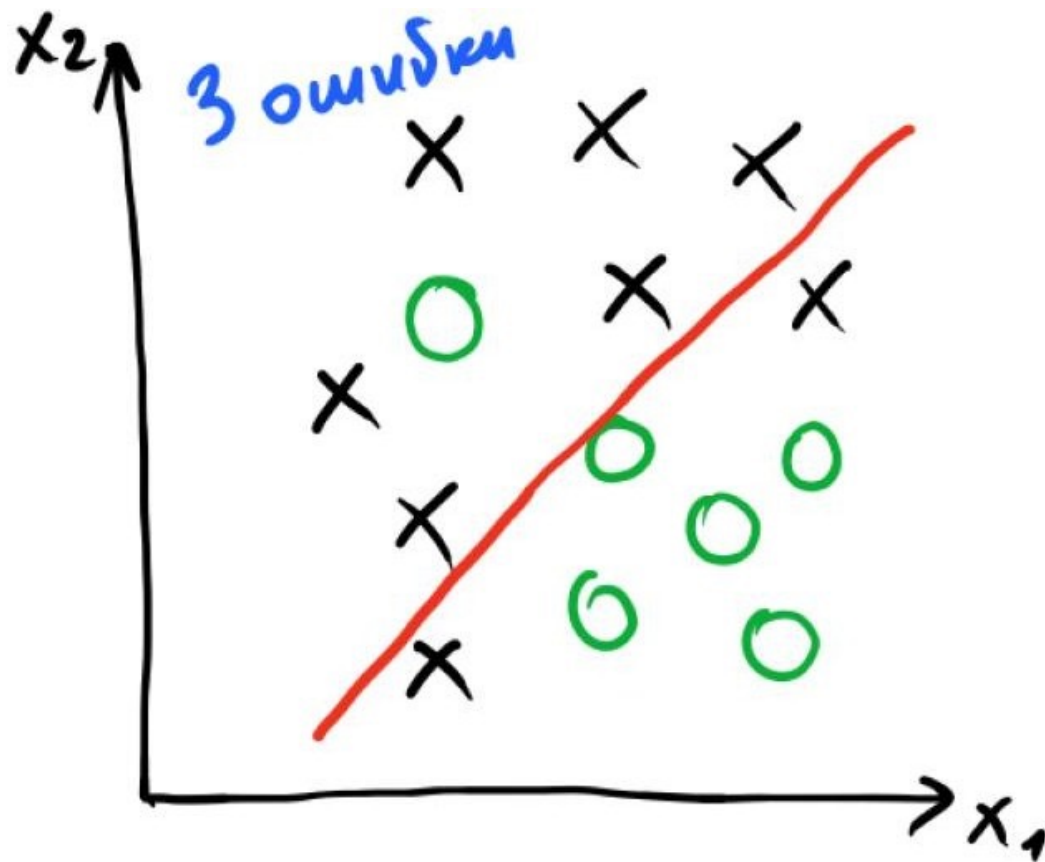


Каждая вершина — вопрос с ответом «да / нет», каждый лист — готовое решение.

Пример



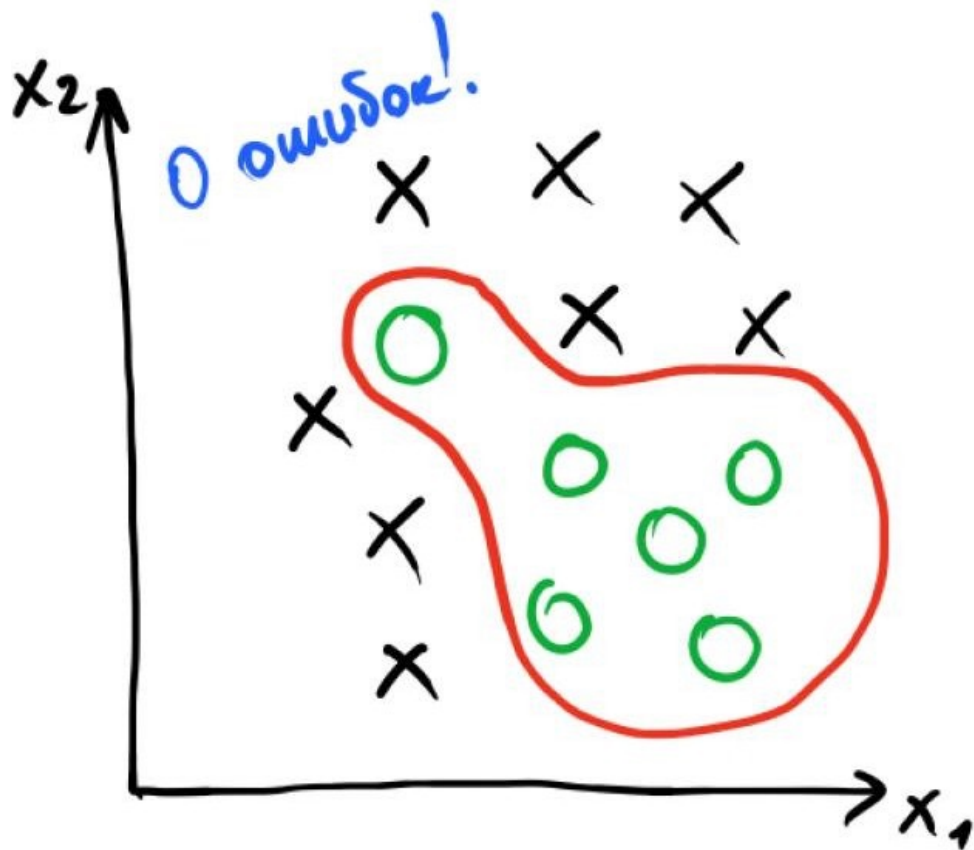
Линейная модель



Три ошибки

Лучшая прямая всё равно отрезает часть ноликов вместе с крестиками.

Нелинейный алгоритм



Ноль ошибок

Если разрешить границе быть произвольной формы, выборку можно разделить идеально. Осталось научиться строить такую границу автоматически.

Решающее дерево

Решающее дерево — это бинарное дерево, в котором:

- 1) каждой вершине v приписана функция (предикат)

$$\beta_v : X \rightarrow \{0, 1\}$$

- 2) каждой листовой вершине v приписан прогноз

$$c_v \in Y$$

для классификации — класс или вероятность класса, для регрессии — действительное значение целевой переменной.



Жадный алгоритм построения

Шаг 1: найти наилучшее разбиение всей выборки на две части

$$R_1(j, t) = \{ x \mid x_j < t \}, \quad R_2(j, t) = \{ x \mid x_j \geq t \}$$

с точки зрения некоторого функционала качества $Q(R, j, t)$:

- Найдём наилучшие j и t — номер признака и порог
- Создадим корень дерева, поставив в него предикат $[x_j < t]$

Перебор ведётся по всем признакам j и всем осмысленным порогам t . На каждом шаге выбираем лучший ход здесь и сейчас, не заглядывая вперёд.

Жадный алгоритм построения

Шаг 2: Рекурсия

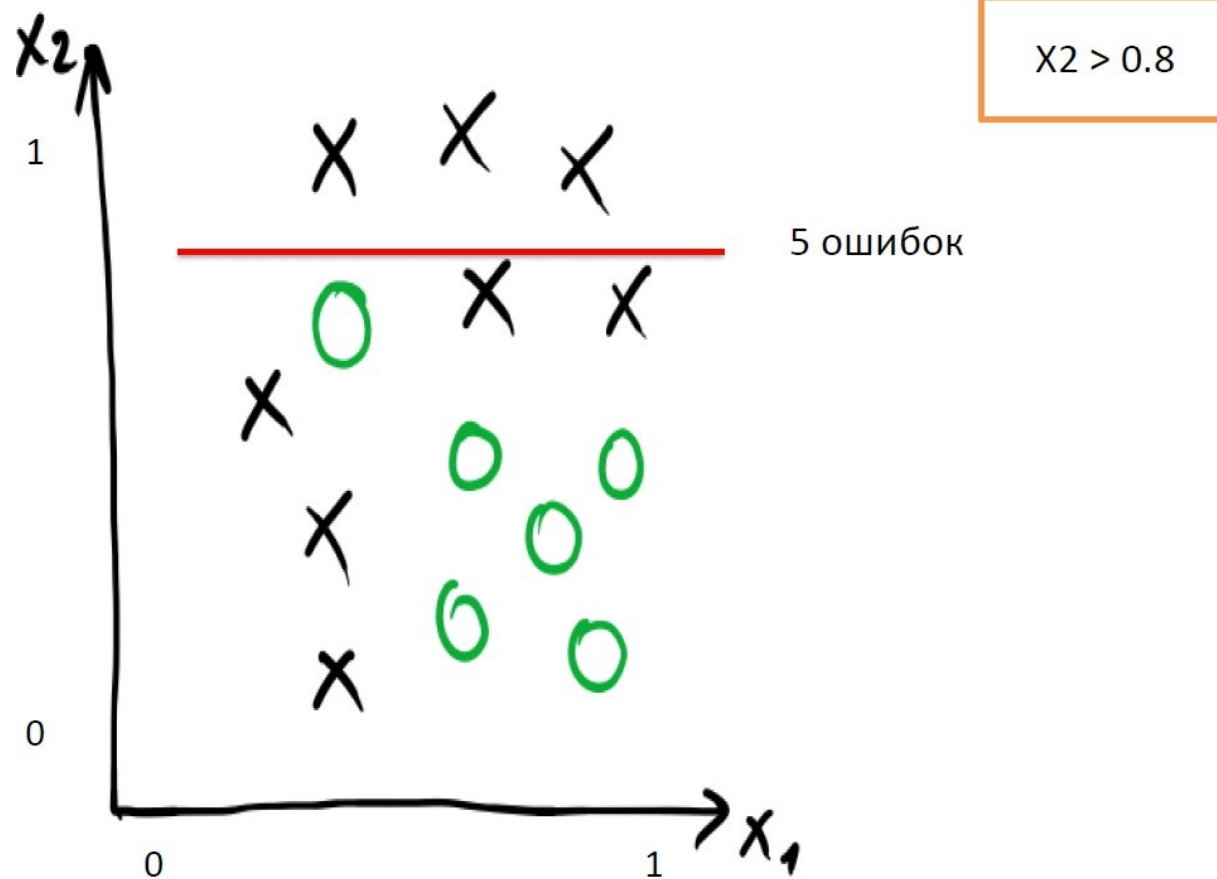
Для каждой из полученных подвыборок R_1 и R_2 рекурсивно применим шаг 1. И так далее.

В каждой вершине на каждом шаге проверяем, не выполнилось ли условие останова. Если выполнилось — объявляем вершину листом и записываем в него предсказание.

- 1 **Разбить** — перебрать (j, t) и выбрать лучшее разбиение текущей подвыборки
- 2 **Проверить останов** — если условие выполнено — вершина становится листом
- 3 **Повторить** — применить то же самое к левой и правой подвыборке

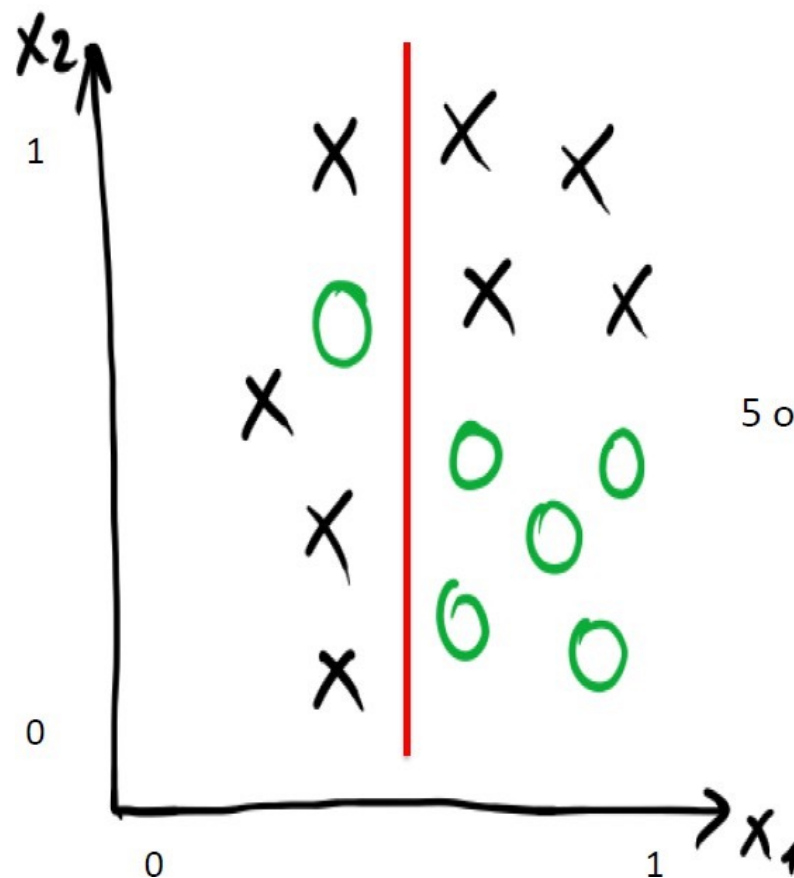
Пример: перебираем предикаты

Предикат $x_2 > 0,8$ — пять ошибок



Пример: перебираем предикаты

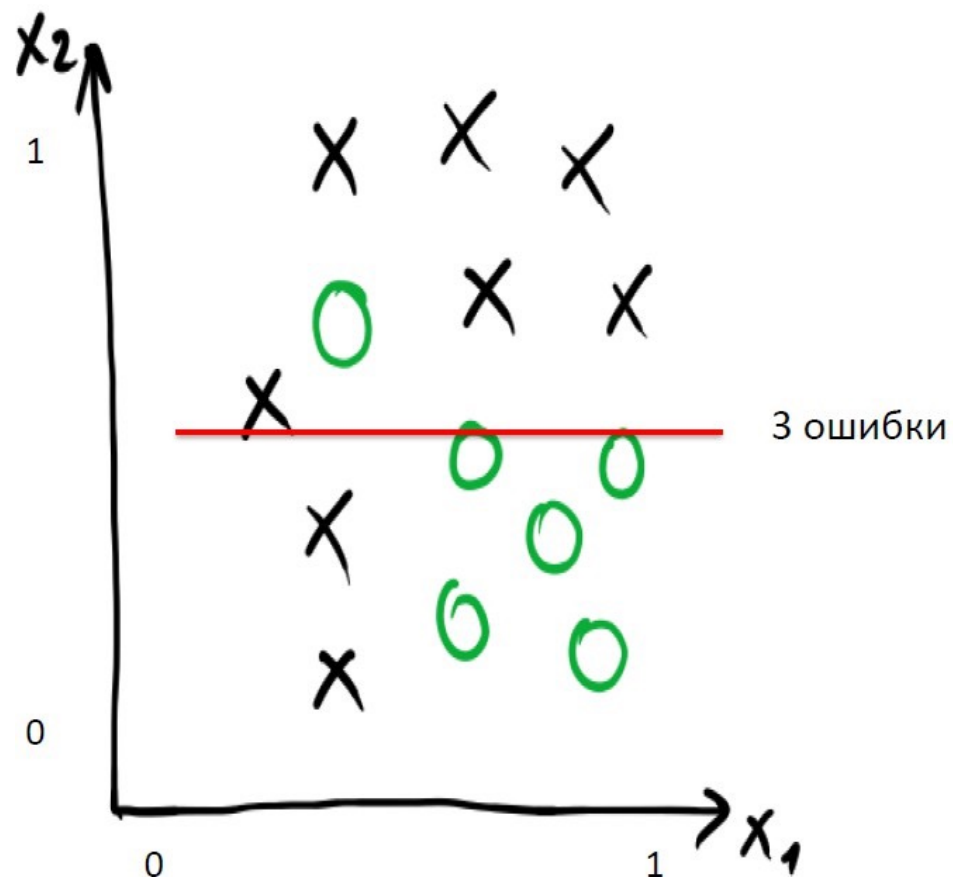
Предикат $x_1 > 0,5$ — снова пять ошибок



$$x_1 > 0.5$$

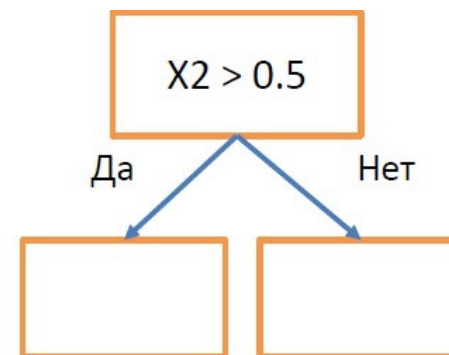
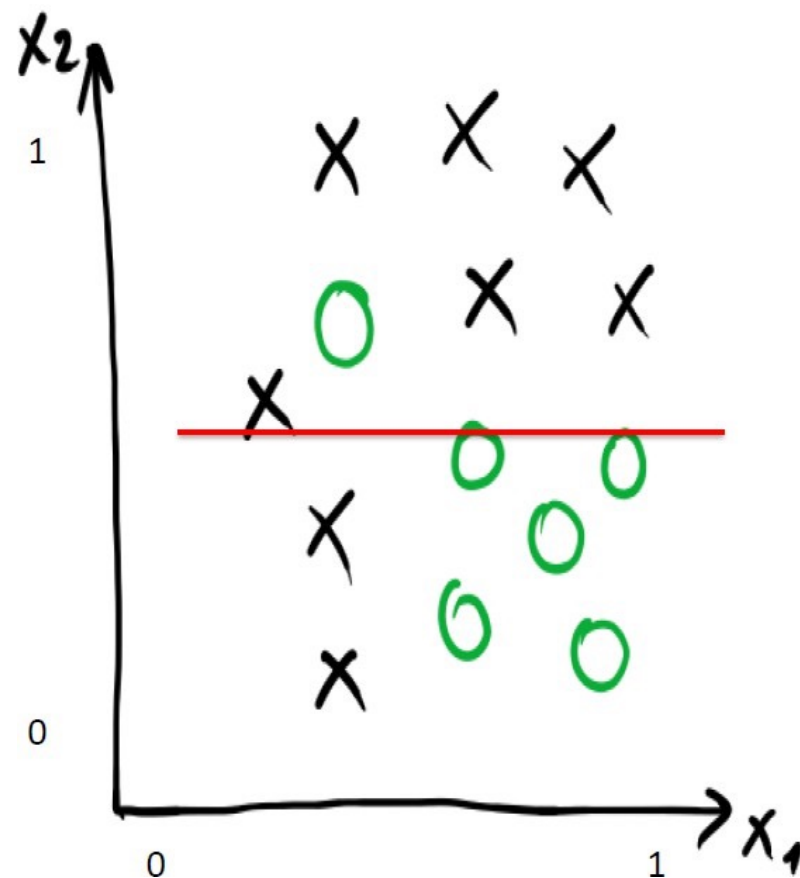
Пример: перебираем предикаты

Предикат $x_2 > 0,5$ — всего три ошибки, это лучше



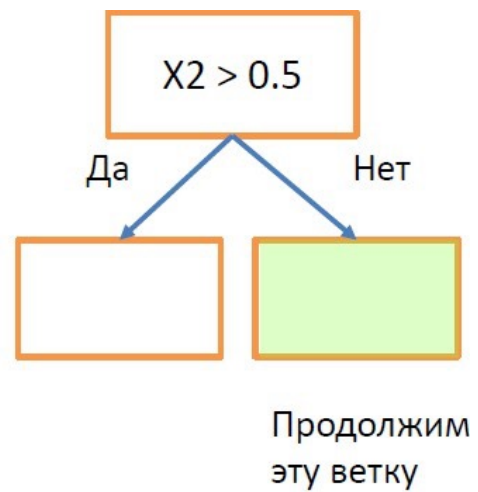
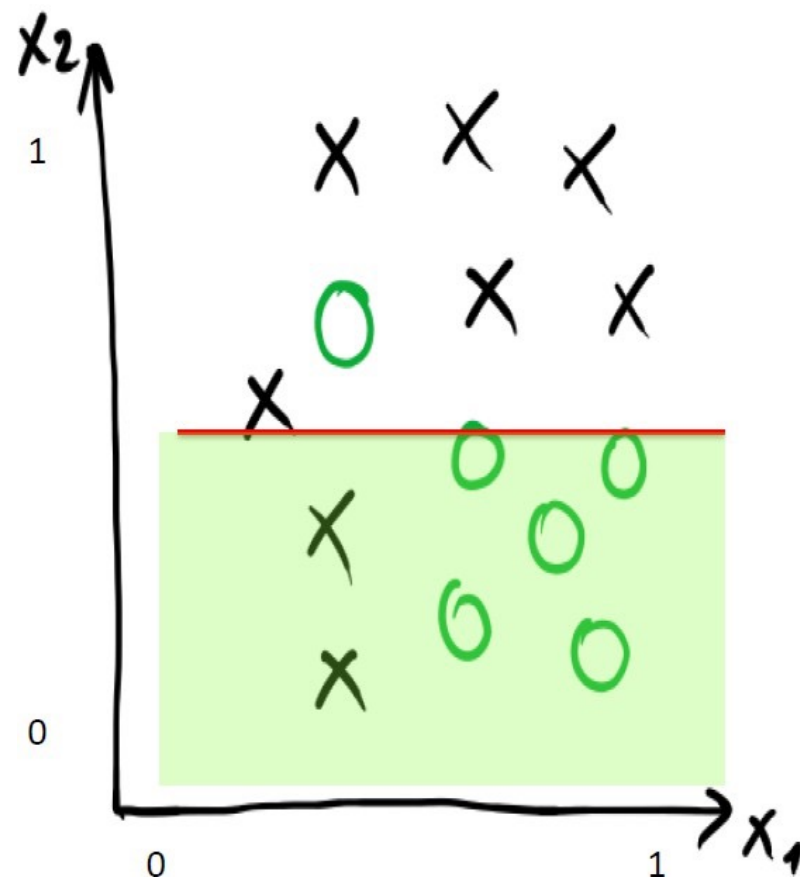
Нашли лучшее первое ветвление

Корень дерева: $X_2 > 0,5$



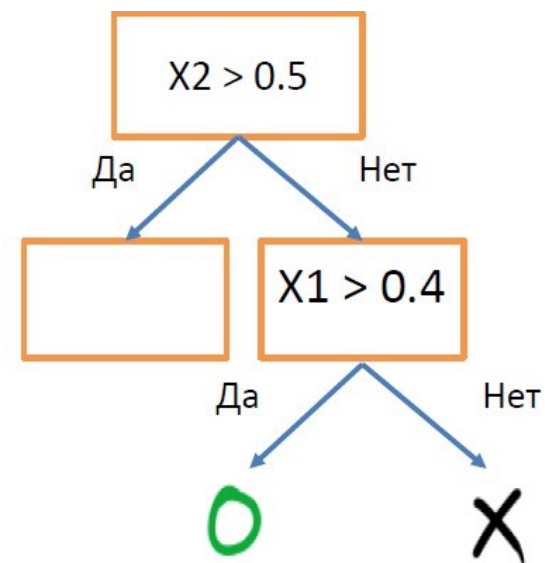
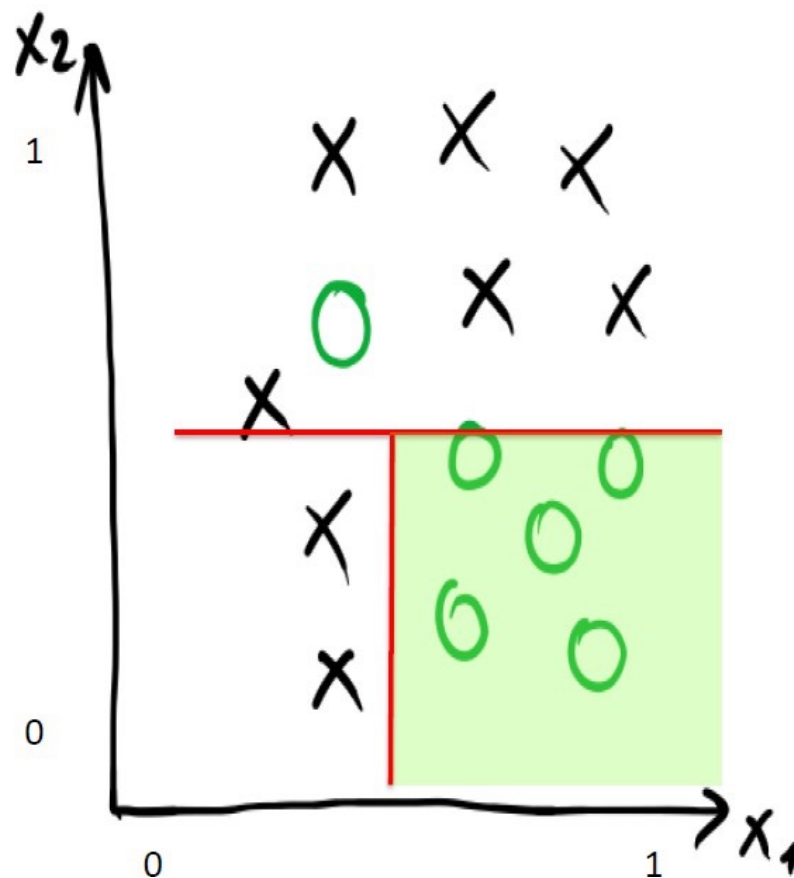
Продолжаем строить

Продолжаем правую ветку



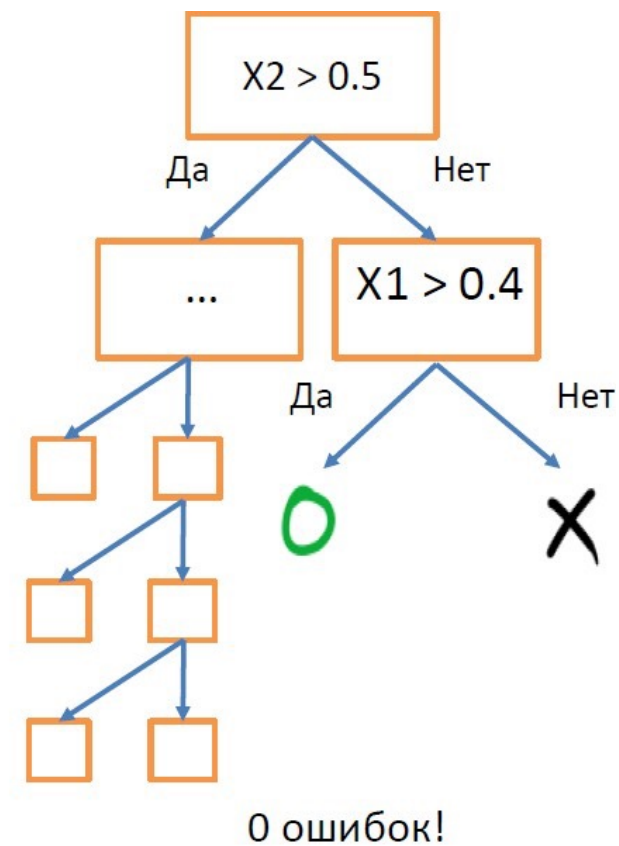
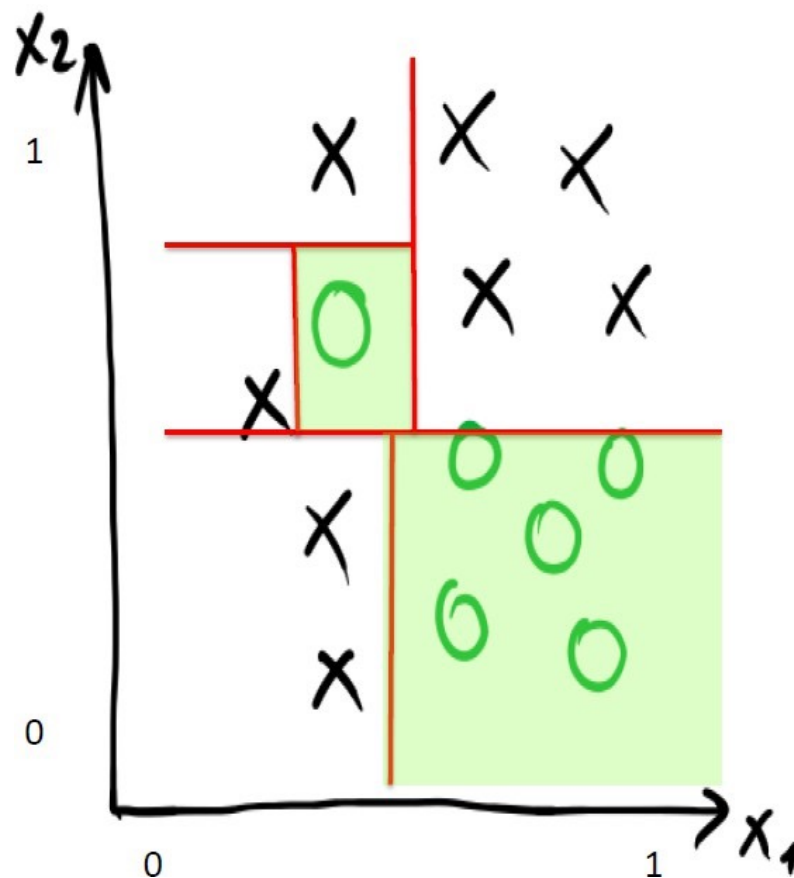
Нашли лучшее второе ветвление

Внутри нижней подвыборки лучший предикат — $X_1 > 0,4$

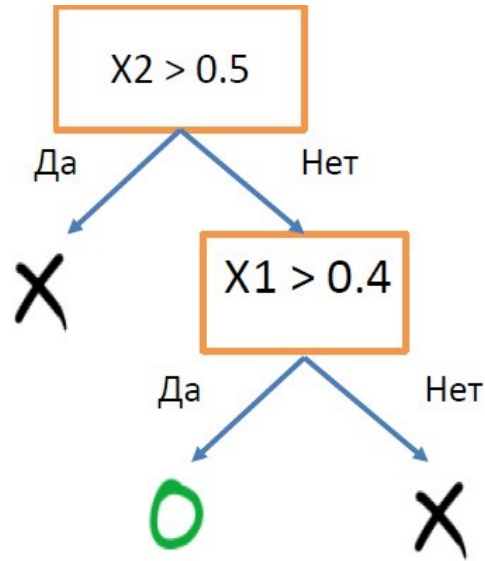
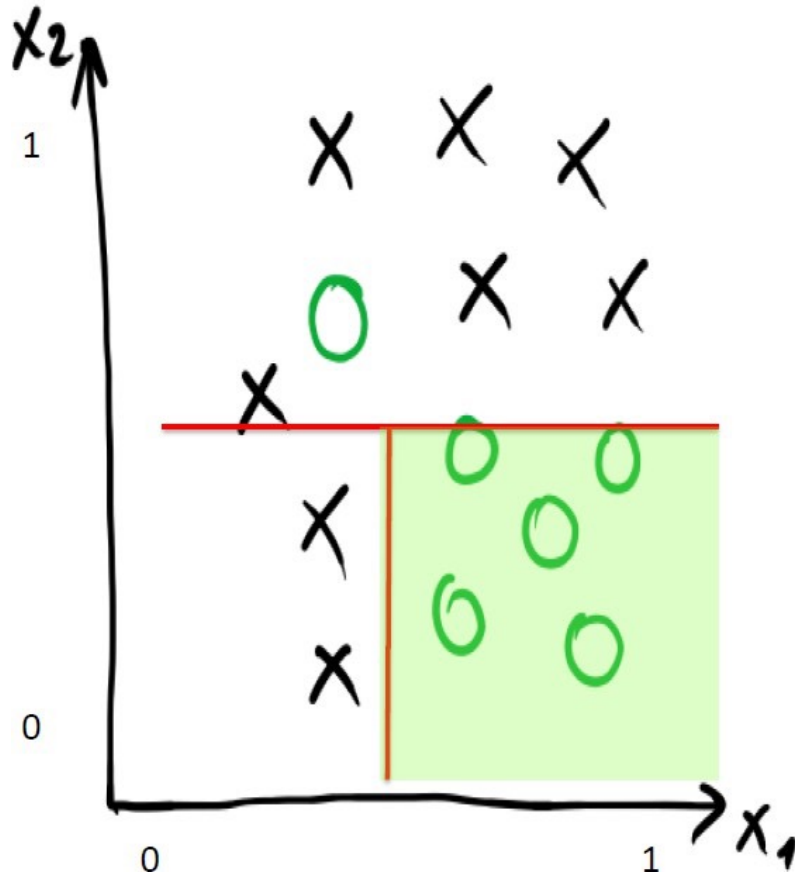


Построили всё дерево

Каждый лист однороден: ноль ошибок на обучающей выборке



Переобучение



1 ошибка, но
скорее всего будет
лучше на тесте!

- Для любой выборки можно построить решающее дерево, не допускающее на ней ни одной ошибки. Такое дерево скорее всего будет переобученным.
- Дерево с одной ошибкой на обучении, но более простое, скорее всего окажется лучше на тесте.

Что влияет на построение решающего дерева

- 1 **Вид предикатов в вершинах** — сравнение с порогом, линейная комбинация признаков и т. д.
- 2 **Функционал качества $Q(X, j, t)$** — по нему выбирается разбиение в каждой вершине
- 3 **Критерий останова** — когда перестать дробить и объявить вершину листом
- 4 **Метод обработки пропущенных значений** — что делать, если признак неизвестен
- 5 **Метод стрижки** — как упростить уже построенное дерево

Дальше разберём каждый из этих пунктов по отдельности.

Критерии информативности

В каждой вершине оптимизируем функционал $Q(X, j, t)$

Пусть R — множество объектов, попадающих в вершину на данном шаге, а R_l и R_r — объекты, попадающие в левую и правую ветки после разбиения.

Цель

Хотим, чтобы после разбиения объектов на две группы внутри каждой группы как можно больше объектов было одного класса.

$H(R)$ — критерий информативности — мера неоднородности (разнообразия) целевой переменной внутри группы R .
Чем меньше разнообразие целевой переменной внутри группы, тем меньше значение $H(R)$.

$$H(R_\ell) \rightarrow \min, \quad H(R_r) \rightarrow \min$$

Функционал качества

Определим функционал Q по формуле:

$$Q(R, j, t) = H(R) - H(R_\ell) - H(R_r) \rightarrow \max_{j, t}$$

- $H(R)$ — неоднородность до разбиения; она фиксирована и от (j, t) не зависит
- Второе и третье слагаемые — суммарная неоднородность после разбиения
- Разность показывает, насколько разбиение уменьшило разнообразие — её и максимизируем.

Функционал качества

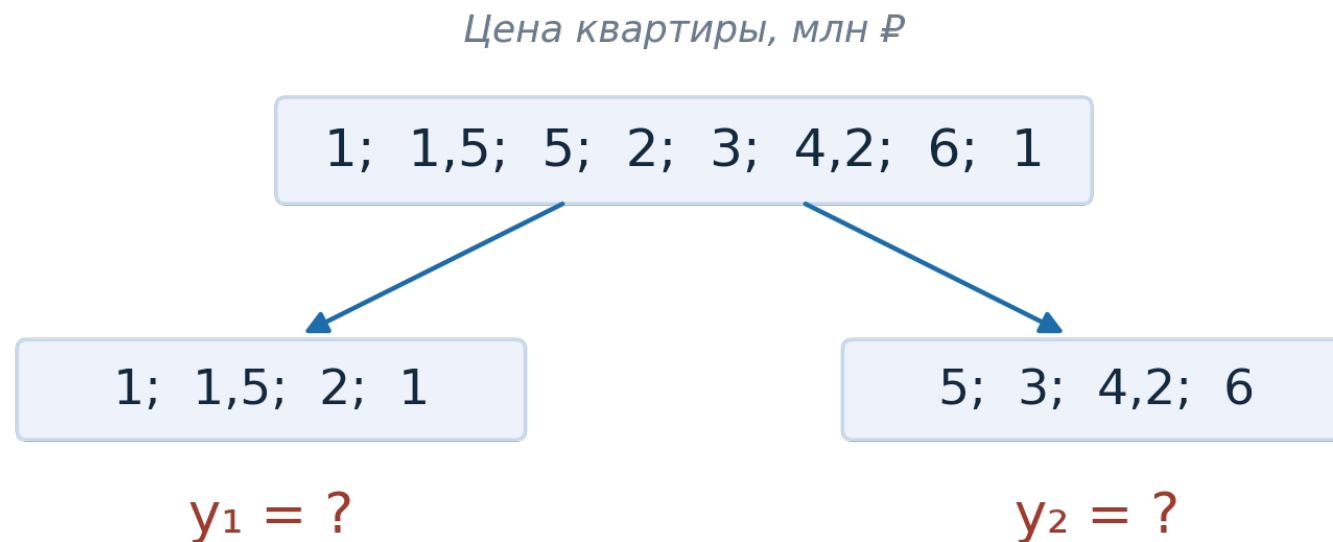
Определим функционал Q по формуле:

$$Q(R, j, t) = H(R) - \frac{|R_\ell|}{|R|} H(R_\ell) - \frac{|R_r|}{|R|} H(R_r) \rightarrow \max_{j, t}$$

- $H(R)$ — неоднородность до разбиения; она фиксирована и от (j, t) не зависит
- Второе и третье слагаемые — средневзвешенная неоднородность после разбиения
- Разность показывает, насколько разбиение уменьшило разнообразие — её и максимизируем.

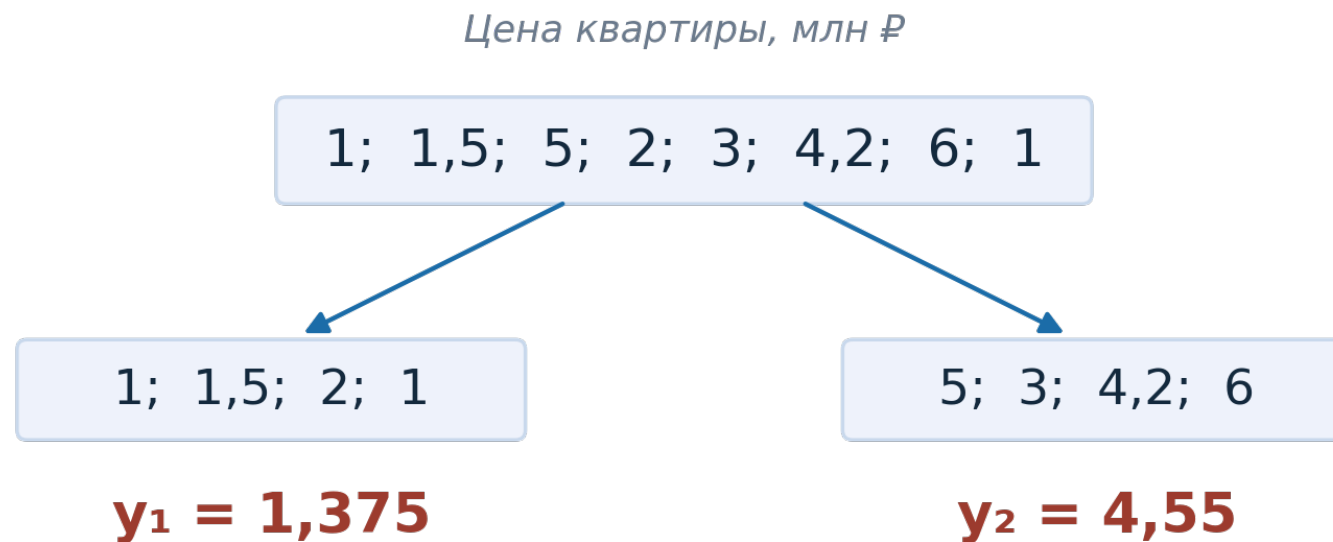
Веса $|R_\ell|/|R|$ и $|R_r|/|R|$ нужны, чтобы маленькая, но идеально чистая ветка не перевешивала большую грязную.

Решающее дерево в задаче регрессии



В каждом листе дерево предсказывает константу. Какую константу выгоднее всего выдать в качестве ответа?

Решающее дерево в задаче регрессии



Если в качестве функционала ошибки в листе использовать среднеквадратичную ошибку, то в качестве ответа надо выдавать среднее значение целевых переменных всех объектов, попавших в лист.

Вид критерия информативности

В каждом листе дерево выдаёт константу c : вещественное число — в регрессии, класс или вероятность класса — в классификации. Чем лучше объекты в листе предсказываются этой константой, тем меньше средняя ошибка на объектах:

$$H(R) = \min_{c \in Y} \frac{1}{|R|} \sum_{(x_i, y_i) \in R} L(y_i, c)$$

где $L(y, c)$ — некоторая функция потерь.

Отсюда дальнейшая логика: выбираем функцию потерь — и критерий информативности получается из неё автоматически.

$H(R)$ в задаче регрессии

Если в качестве функции потерь мы берём квадратичную ошибку, то

$$H(R) = \min_{c \in \mathbb{R}} \frac{1}{|R|} \sum_{(x_i, y_i) \in R} (y_i - c)^2$$

Её минимум достигается при

$$c^* = \frac{1}{|R|} \sum_{(x_i, y_i) \in R} y_i$$

то есть в листе предсказывается среднее значение целевой переменной на объектах, попавших в лист.

Н(R) в задаче регрессии

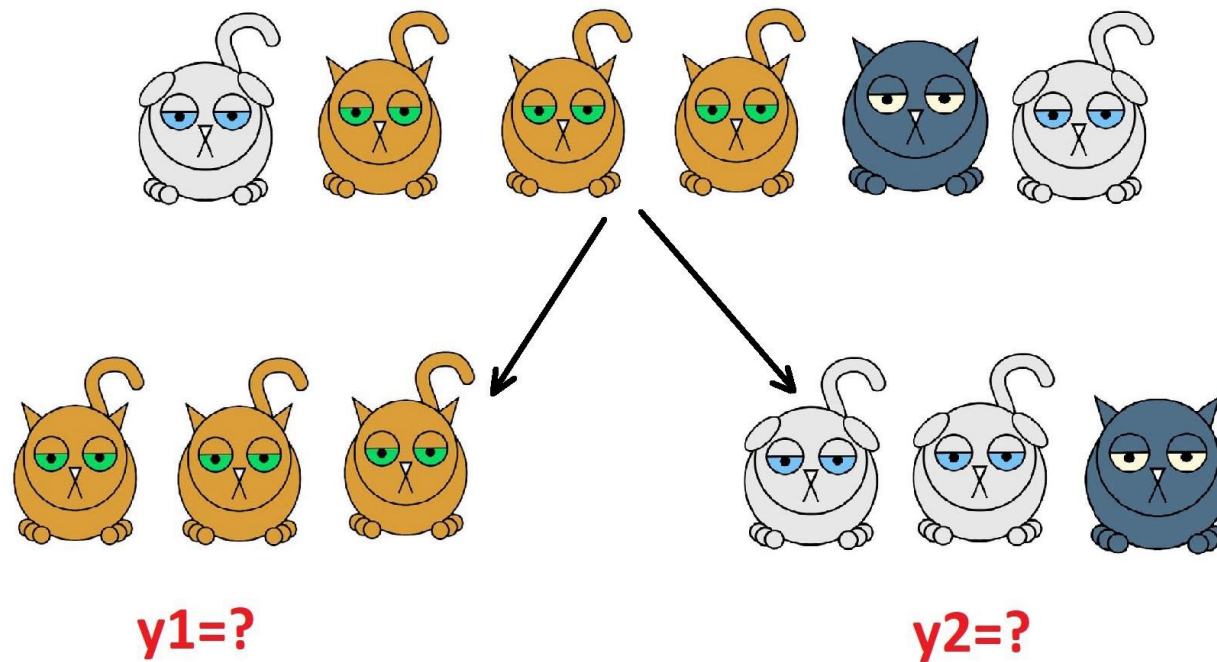
Подставим оптимальную константу обратно

$$H(R) = \frac{1}{|R|} \sum_{(x_i, y_i) \in R} (y_i - \bar{y})^2$$

Информативность $H(R)$ в листе — это дисперсия целевой переменной для объектов, попавших в этот лист. Чем меньше дисперсия, тем меньше разброс целевой переменной объектов в листе.

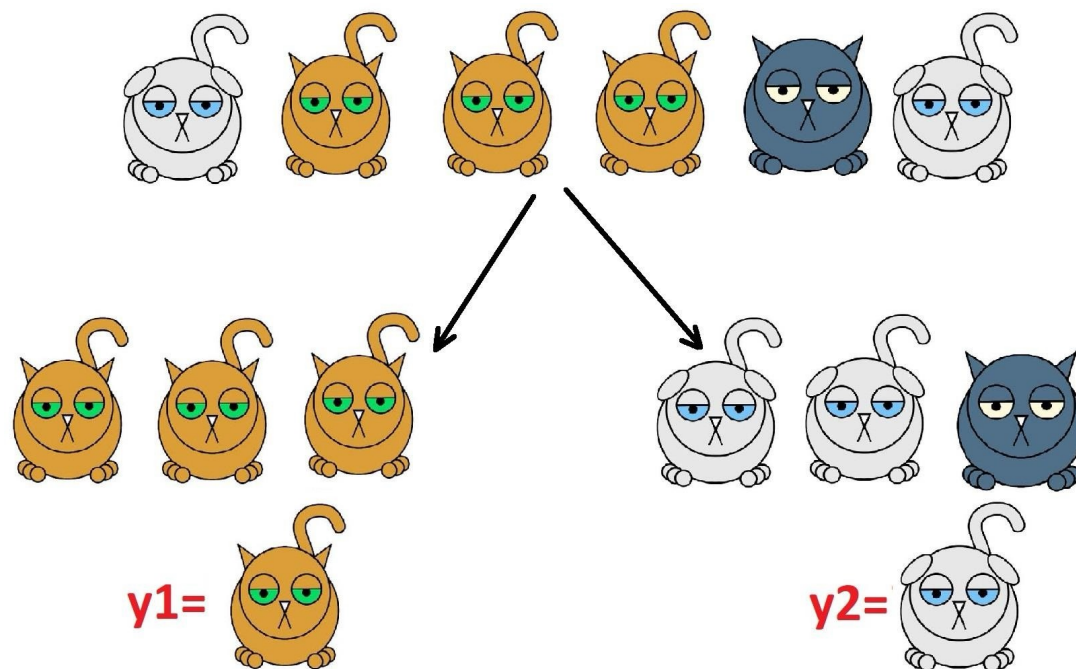
Именно этот критерий в *scikit-learn* называется `criterion="squared_error"` и стоит по умолчанию для *DecisionTreeRegressor*.

Решающее дерево в задаче классификации



В каждом листе дерево предсказывает класс объекта. Какой класс выгоднее всего выдать в качестве ответа?

Решающее дерево в задаче классификации



Разумнее всего в качестве ответа в листе выдавать самый представительный класс — тот, объектов которого в листе больше всего.

Формализация: доли классов

Решаем задачу классификации с K классами: $1, 2, \dots, K$. Пусть p_k — доля объектов класса k , попавших в вершину:

$$p_k = \frac{1}{|R|} \sum_{(x_i, y_i) \in R} [y_i = k]$$

Все критерии информативности для классификации записываются через эти доли p_k .

Ошибка классификации

Пусть k^* — самый представительный класс в данной вершине:

$$k^* = \arg \max_k p_k$$

Если выдавать в листе именно его, доля ошибок в вершине равна

$$H(R) = 1 - p_{k^*}$$

Почему этого мало

Критерий не различает распределения (0,5; 0,3; 0,2) и (0,5; 0,5; 0) — обе дают ошибку 0,5, хотя вторая вершина заметно «чище». Поэтому на практике используют более чувствительные критерии.

Критерий Джини

Будем в каждой вершине в качестве ответа выдавать не класс, а распределение вероятностей классов $c = (c_1, \dots, c_K)$, где сумма всех компонент равна 1. Качество распределения измерим с помощью критерия Бриера:

$$H(R) = \min_c \frac{1}{|R|} \sum_{(x_i, y_i) \in R} \sum_{k=1}^K ([y_i = k] - c_k)^2$$

Утверждение. Минимальное значение функционала $H(R)$ достигается на векторе, состоящем из долей классов:

$$c^* = (p_1, \dots, p_K)$$

Критерий Джини

Будем в каждой вершине в качестве ответа выдавать не класс, а распределение вероятностей классов $c = (c_1, \dots, c_K)$, где сумма всех компонент равна 1. Качество распределения измерим с помощью критерия Бриера:

$$H(R) = \min_c \frac{1}{|R|} \sum_{(x_i, y_i) \in R} \sum_{k=1}^K ([y_i = k] - c_k)^2$$

Утверждение. Минимальное значение функционала $H(R)$ достигается на векторе, состоящем из долей классов:

$$c^* = (p_1, \dots, p_K)$$

На этом векторе получаем

$$H(R) = \sum_{k=1}^K p_k(1 - p_k)$$

критерий Джини

Энтропийный критерий

Возьмём вместо квадратичной ошибки логарифм правдоподобия:

$$H(R) = \min_c \left[-\frac{1}{|R|} \sum_{(x_i, y_i) \in R} \sum_{k=1}^K [y_i = k] \log c_k \right]$$

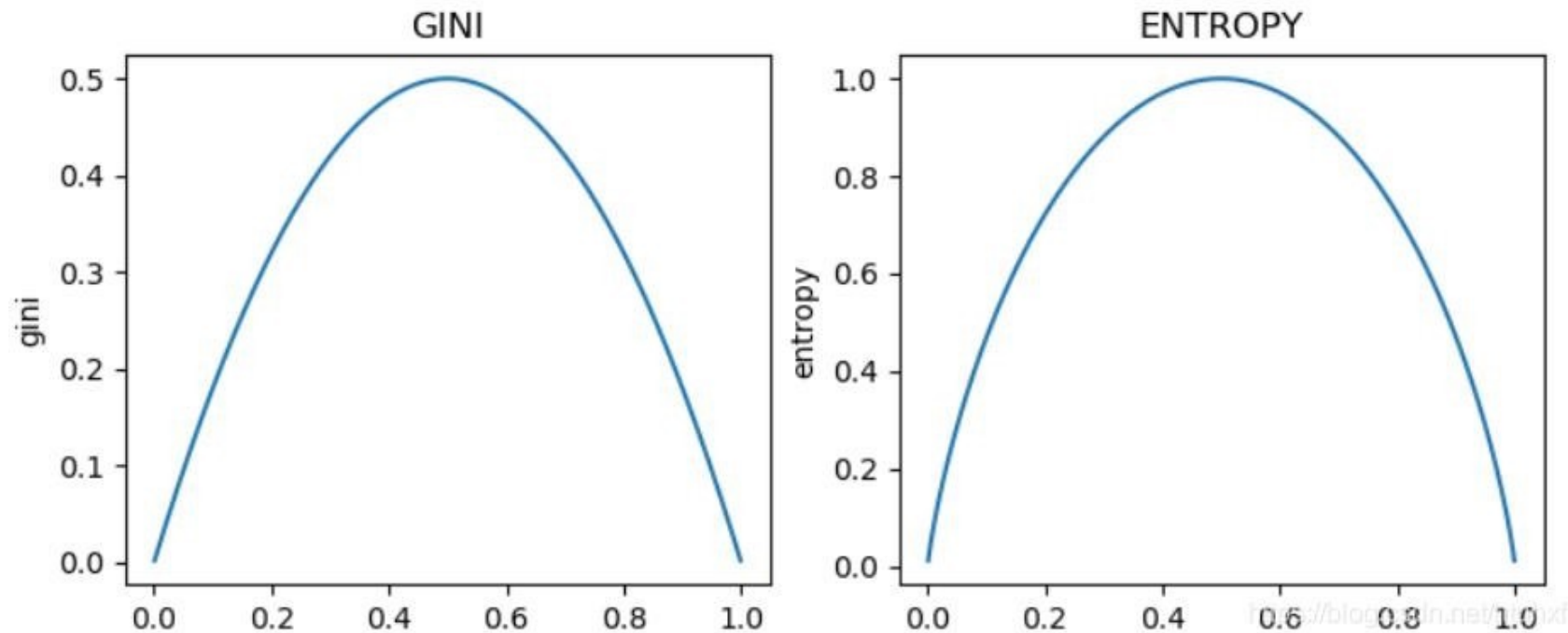
На том же векторе $c^* = (p_1, \dots, p_K)$ функционал записывается в виде

$$H(R) = - \sum_{k=1}^K p_k \log p_k$$

— это энтропия распределения классов в вершине.

Критерии Gini и Entropy

Обе функции максимальны при $p = 0,5$ и обращаются в ноль на чистой вершине



Графики для бинарного случая: по горизонтали — доля объектов первого класса.

Свойства энтропии

Минимум: $H(R) = 0$

$$p_i = 1, \quad p_j = 0 \quad (j \neq i)$$

Все объекты вершины принадлежат одному классу — вершина идеально чистая.

Максимум

$$p_1 = \dots = p_K = \frac{1}{K}$$

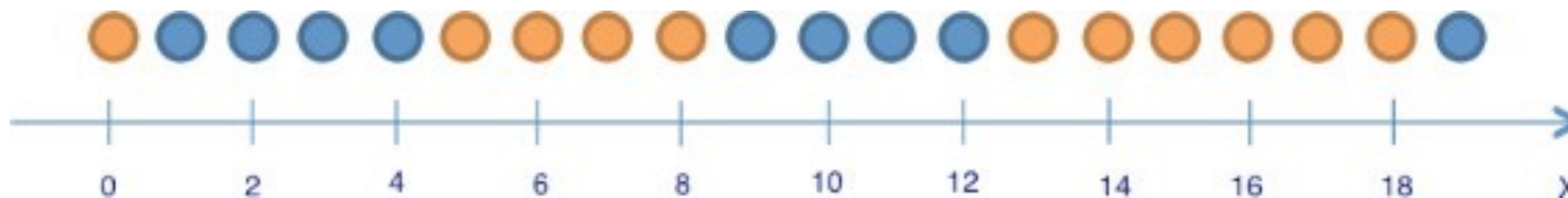
Достигается на равномерном распределении: классы перемешаны поровну, вершина максимально неоднородна.

Что выбрать на практике

Энтропия и критерий Джини дают очень похожие деревья. Джини считается чуть быстрее (нет логарифма) и стоит в scikit-learn по умолчанию; энтропия чуть сильнее штрафует неоднородные вершины.

Пример использования энтропийного критерия

Шаг 1: считаем энтропию в корне

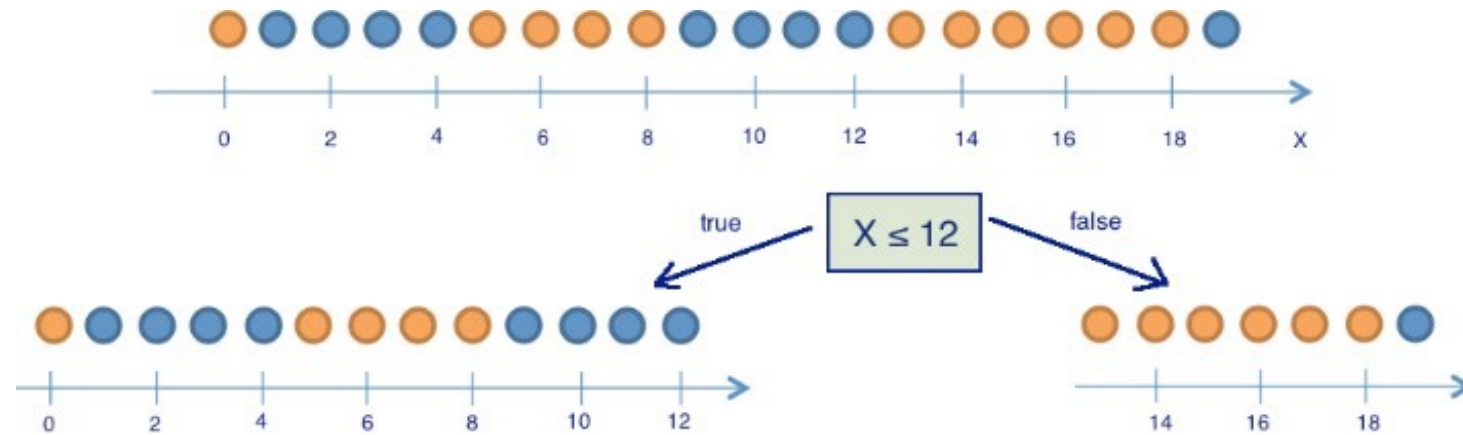


$$p_1 = \frac{9}{20}, \quad p_2 = \frac{11}{20} \Rightarrow H(R) = -\frac{9}{20} \log \frac{9}{20} - \frac{11}{20} \log \frac{11}{20} \approx 1$$

Двадцать объектов, девять одного класса и одиннадцать другого — распределение почти равномерное, энтропия близка к максимуму.

Пример использования энтропийного критерия

Шаг 2: разбиваем предикатом $X \leq 12$



$$H(R_\ell) = -\frac{5}{13} \log \frac{5}{13} - \frac{8}{13} \log \frac{8}{13} \approx 0,96$$

$$H(R_r) = -\frac{1}{7} \log \frac{1}{7} - \frac{6}{7} \log \frac{6}{7} \approx 0,6$$

$$Q = 1 - \frac{13}{20} \cdot 0,96 - \frac{7}{20} \cdot 0,6 \approx 0,16$$

Критерии останова

- Ограничение максимальной глубины дерева (`max_depth`)
- Ограничение минимального числа объектов в листьях (`min_samples_leaf`)
- Ограничение минимального числа объектов для разбиения (`min_samples_split`)
- Ограничение максимального числа листьев в дереве (`max_leaf_nodes`)
- Останов в случае, если все объекты в листе из одного класса
- Требование, чтобы функционал качества при дроблении увеличивался как минимум на s %

Все эти параметры — гиперпараметры: их подбирают на отложенной выборке или кросс-валидацией, а не по обучающим данным.

Стрижка дерева (pruning)

Альтернатива критериям останова

- 1 **Строим переобученное дерево** — дробим до упора, в каждом листе по одному объекту
- 2 **Оптимизируем его структуру** — удаляем ветви, которые не улучшают качество на отложенной выборке

В чём разница

Критерий останова решает судьбу ветки, ещё не увидев, что под ней. Стрижка принимает решение, уже зная всё дерево целиком, — и потому обычно работает точнее.



Cost-complexity pruning

Пусть $R(T)$ — ошибка на дереве T . Введём регуляризованный функционал:

$$R_{\alpha}(T) = R(T) + \alpha |T|$$

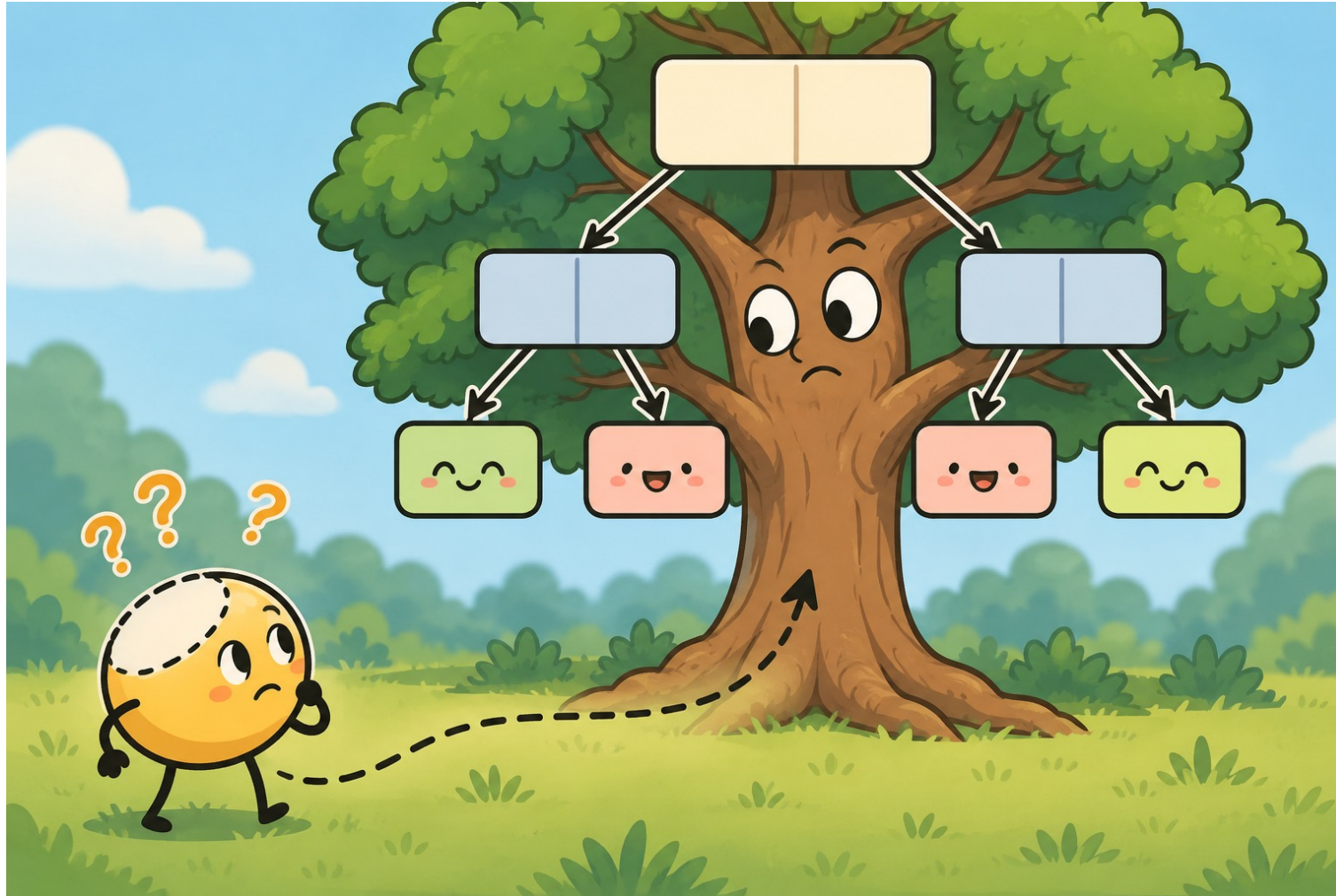
где $|T|$ — количество листьев в дереве.

При построении дерева и оптимизации функционала $R_{\alpha}(T)$ дерево не будет иметь большое количество вершин, а потому будет менее переобученным.

*Параметр α задаёт цену одной вершины: при $\alpha = 0$ получаем исходное переобученное дерево, при больших α — вырожденное дерево из одного листа. В *scikit-learn* это параметр `ccp_alpha`.*

Обработка пропущенных значений

Как выбрать предикат, если часть значений неизвестна



Обработка пропущенных значений

Как выбрать предикат, если часть значений неизвестна

Пусть есть некоторый предикат $\vartheta(x) = [x_j < t]$, но в выборке R есть объекты, для которых неизвестно значение признака x_j :

- Обозначим их V_j
- Тогда **функционал качества можно вычислять, просто игнорируя объекты из V_j :**

$$Q(R, j, t) = \frac{|R \setminus V_j|}{|R|} \left(H(R \setminus V_j) - \frac{|R_\ell|}{|R \setminus V_j|} H(R_\ell) - \frac{|R_r|}{|R \setminus V_j|} H(R_r) \right)$$

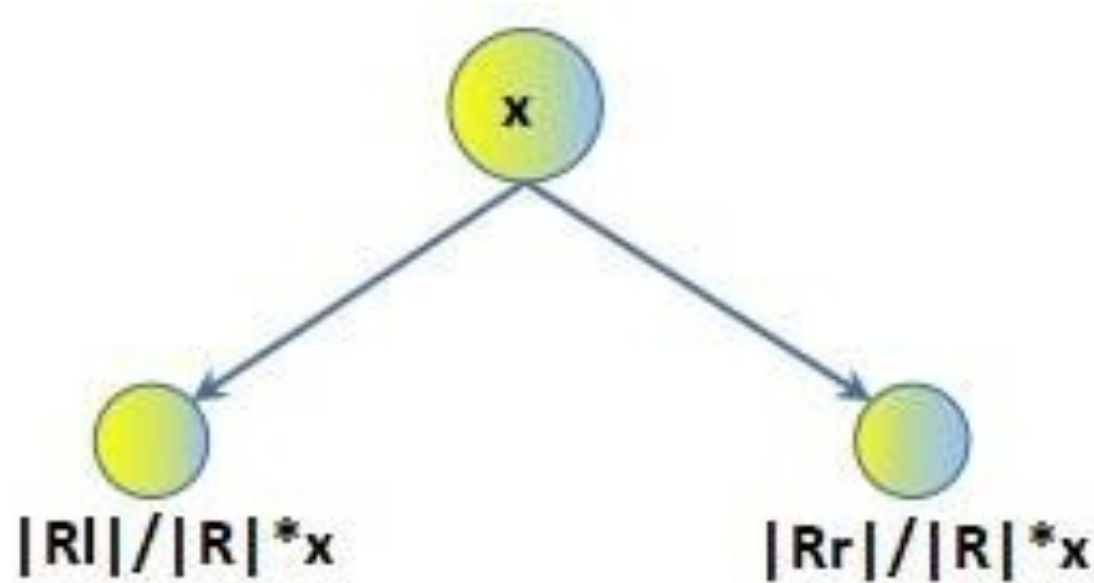
Далее выбираем наилучший предикат по данному функционалу качества.

Множитель $|R \setminus V_j| / |R|$ штрафует признаки с большим числом пропусков: чем меньше объектов, на которых предикат вообще определён, тем ниже его шанс попасть в вершину.

Получение предсказаний для пропущенных значений

Куда отправить объекты с пропуском

Поместим объекты из V_j и в левое, и в правое поддереву после разбиения предикатом ϑ . Присвоим этим объектам веса $\frac{|R_\ell|}{|R|}$ и $\frac{|R_r|}{|R|}$



Объект с пропуском как бы «размазывается» по обеим веткам пропорционально тому, сколько обычных объектов туда ушло. При подсчёте критериев информативности такие объекты учитываются со своими весами.

Получение предсказаний для пропущенных значений

Если объект попал в вершину, предикат в которой для него не может быть вычислен из-за пропуска, то он попадает в оба поддерева с весами, и ответы в обоих поддеревьях затем усредняются.

Обозначим прогноз вероятности класса k для объекта x в поддереве R_m как $a_{mk}(x)$. Тогда:

$$a_k(x) = \sum_{m \in \{\ell, r\}} \frac{|R_m|}{|R|} a_{mk}(x)$$

Дерево не отказывается предсказывать и не требует заранее заполнять пропуски: оно честно усредняет два возможных ответа с теми же весами, что использовались при обучении.

Другие способы обработки пропущенных значений

Метод суррогатных предикатов

- Если есть объекты, для которых невозможно вычислить значение предиката $[x_j < t]$, то находим другой предикат, который даёт максимально похожее разбиение на R_l и R_r .
- Такой предикат хранится в вершине как запасной и используется только при пропуске.

Заполнение специальным значением

- Можно заменить все пропущенные значения на 0 или на какое-то уникальное число (например, -999).
- Тогда можно будет использовать обычную процедуру построения дерева: пропуски просто уедут в одну из веток.

Второй способ грубее, но работает с любой реализацией дерева и часто оказывается достаточным на практике.

Обработка категориальных признаков

Multi-way splits



Обработка категориальных признаков

Multi-way splits

Можно разбивать вершину на столько поддеревьев, сколько различных значений имеет категориальный признак.

Получаем очень большие деревья: признак с сотней категорий даёт сто веток, в каждой из которых остаётся слишком мало объектов.

Поэтому обычно поступают иначе. Поделим множество возможных значений признака $Q = \{u_1, \dots, u_m\}$ на два подмножества и определим предикат:

$$Q = Q_1 \sqcup Q_2, \quad \beta(x) = [x_j \in Q_1]$$

Сколько вариантов нужно перебрать

Для построения оптимального предиката нужно перебрать столько вариантов разбиения $Q = Q_1 \cup Q_2$:

$$2^m - 1 - 1$$

Уже при $m = 20$ категориях это примерно полмиллиона вариантов — и так в каждой вершине по каждому категориальному признаку. Полный перебор неприменим.

К счастью, есть приём, который даёт тот же результат за $O(m \log m)$: **достаточно упорядочить категории по одному числу и перебрать только разбиения на префикс и суффикс.**

Категориальные признаки в бинарной классификации

Рассмотрим вершину дерева. Пусть $N(u_1)$ — количество объектов в этой вершине со значением категориального признака $x_j = u_1$. Тогда

$\frac{N_+(u_1)}{N(u_1)}$ — доля положительных объектов среди объектов с $x_j = u_1$.

Упорядочим значения категориального признака по возрастанию этих долей:

$$\frac{N_+(u_{(1)})}{N(u_{(1)})} \leq \frac{N_+(u_{(2)})}{N(u_{(2)})} \leq \dots \leq \frac{N_+(u_{(m)})}{N(u_{(m)})}$$

Заменяем категорию $u_{(i)}$ на i и будем работать с полученным вещественным признаком.

Утверждение

Данный подход с использованием критерия Джини или энтропийного критерия даёт тот же результат, как если бы мы искали оптимальное разбиение на $Q = Q_1 \cup Q_2$.

То есть полный перебор $2^{m-1} - 1$ вариантов можно заменить одной сортировкой категорий и обычным перебором порогов по вещественному признаку — без потери качества разбиения.

Осторожно: у признаков с большим числом категорий это легко приводит к переобучению — дерево «запоминает» редкие категории.

Категориальные признаки в задачах регрессии

Упорядочим значения категориального признака x_j по среднему ответу на объектах с таким значением:

$$\frac{\sum_{x_{ij}=u_{(1)}} y_i}{N(u_{(1)})} \leq \frac{\sum_{x_{ij}=u_{(2)}} y_i}{N(u_{(2)})} \leq \dots \leq \frac{\sum_{x_{ij}=u_{(m)}} y_i}{N(u_{(m)})}$$

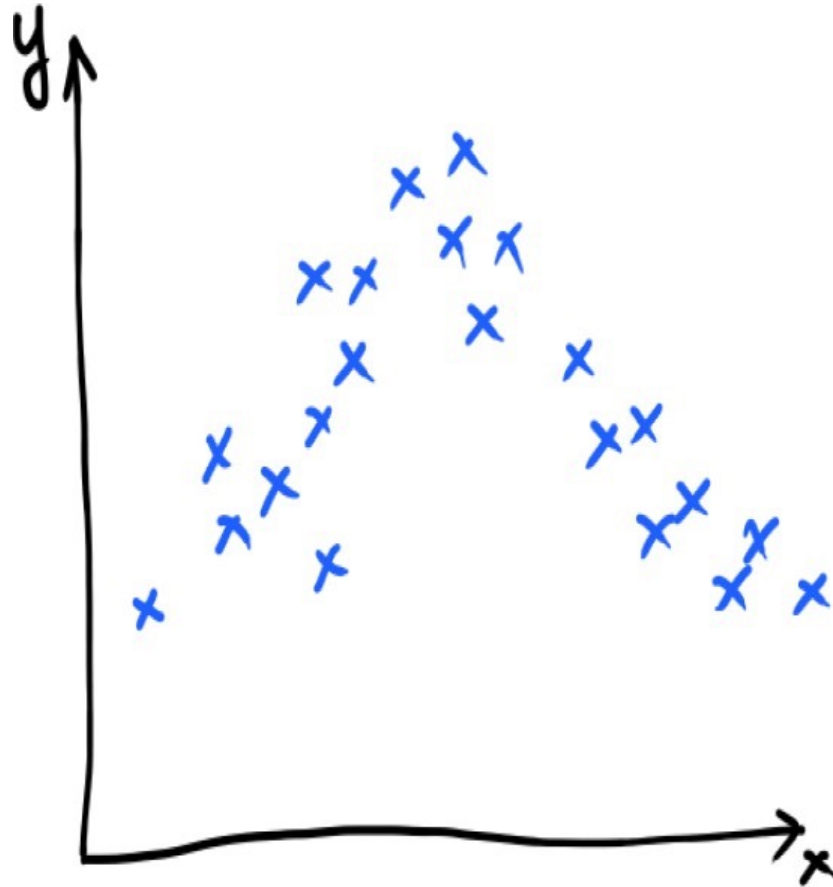
Заменяем категорию $u_{(i)}$ на i и будем работать с полученным вещественным признаком.

Утверждение

Данный подход с использованием MSE-функционала даёт тот же результат, как если бы мы искали оптимальное разбиение на $Q = Q_1 \cup Q_2$.

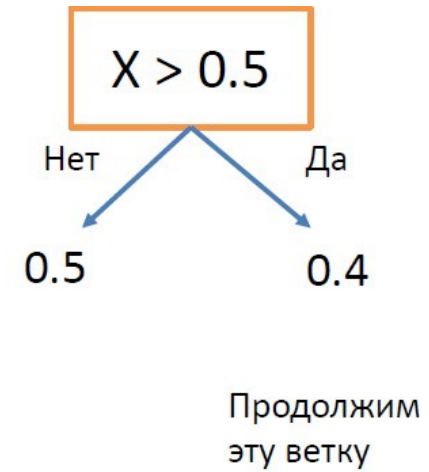
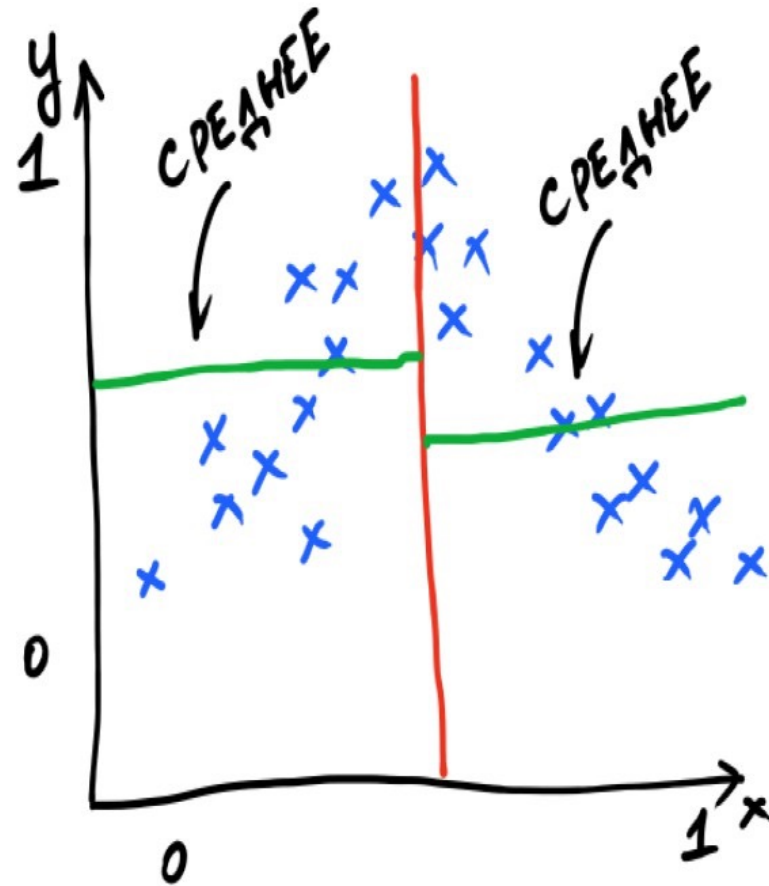
Пример построения дерева в задаче регрессии

Исходные данные: один признак, вещественный ответ



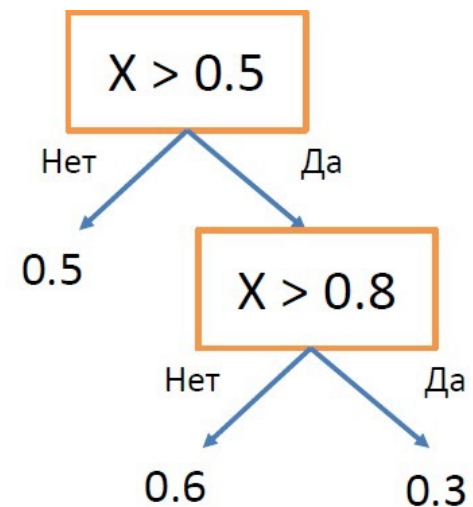
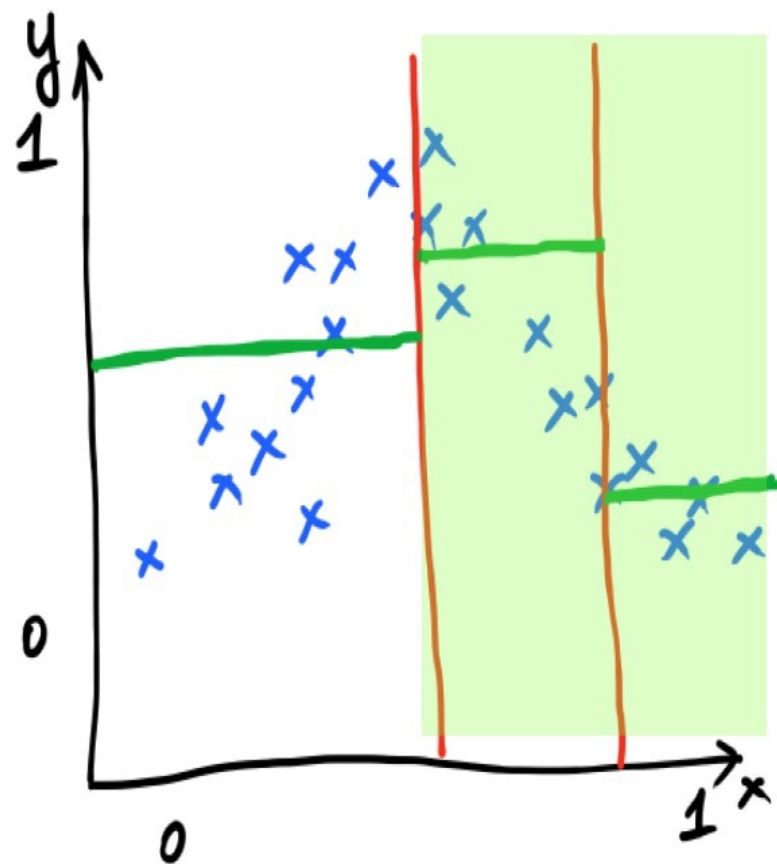
Пример построения дерева в задаче регрессии

Первое разбиение: в каждой половине предсказываем среднее



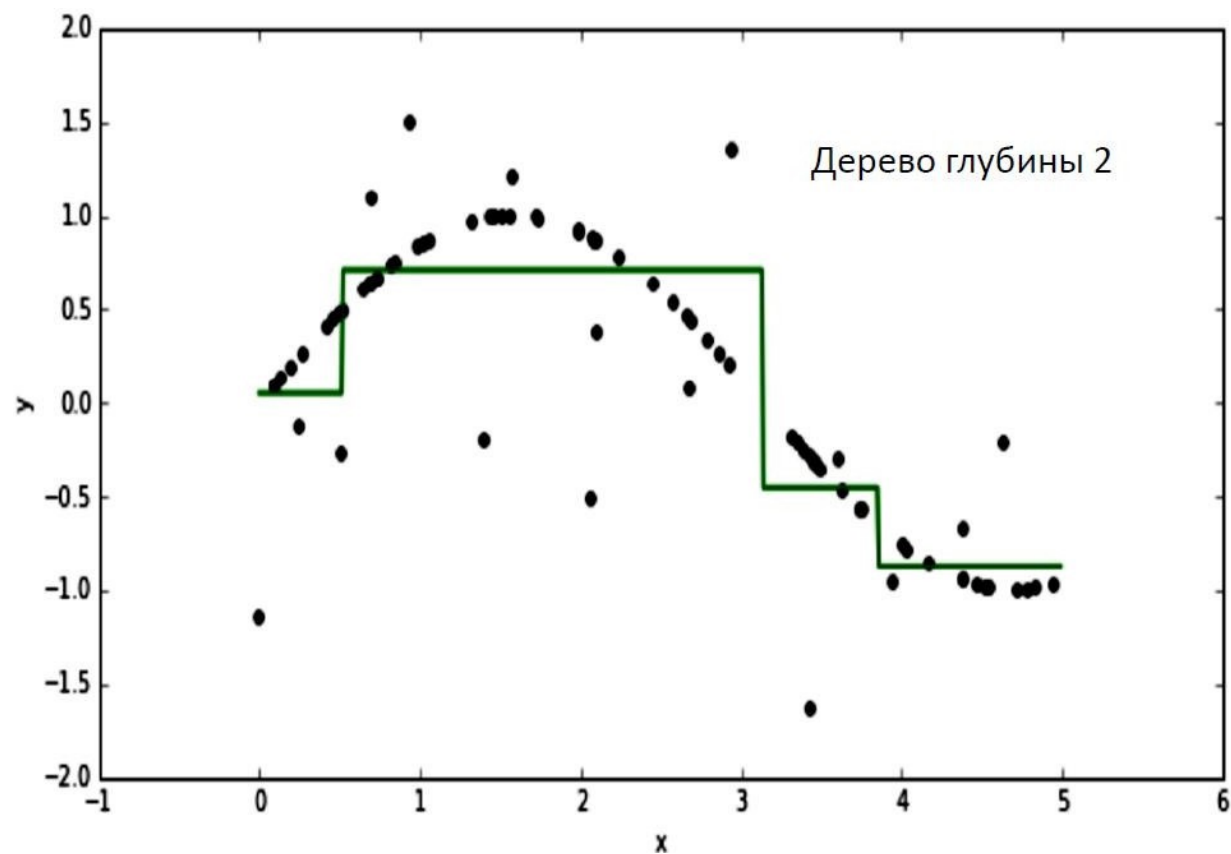
Пример построения дерева в задаче регрессии

Продолжаем ветку: внутри правой части снова ищем лучший порог



Пример: дерево глубины 2

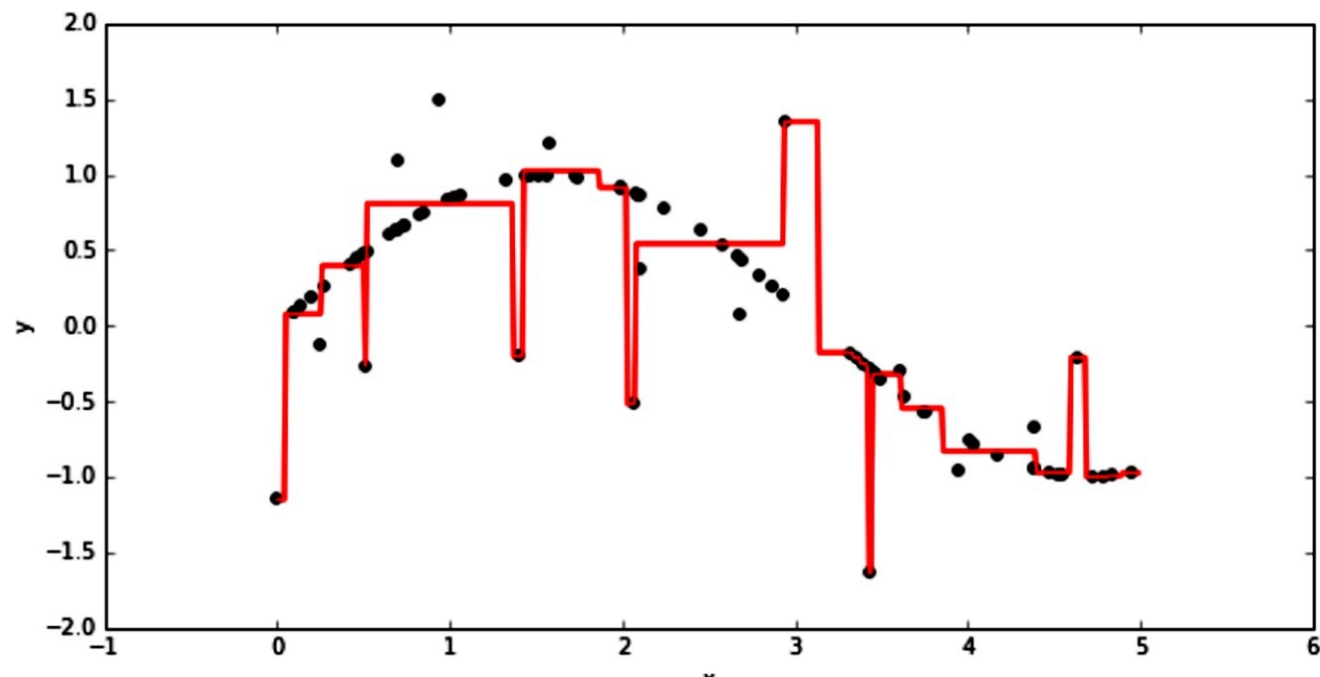
Кусочно-постоянный прогноз из четырёх значений



Дерево небольшой глубины описывает данные грубо, но устойчиво.

Пример: дерево глубины 5

То же самое, но без ограничения сложности



Прогноз начинает повторять отдельные точки — это уже переобучение.

Решающие деревья: итоги

Плюсы

- Чёткие правила классификации: интерпретируемые предикаты, например «возраст > 25»
- Деревья легко визуализируются, то есть хорошо интерпретируются
- Быстро обучаются и выдают прогноз
- Малое число параметров
- Работают с разнотипными признаками, не требуют масштабирования

Минусы

- Очень чувствительны к шумам: модель сильно меняется при небольшом изменении обучающей выборки
- Разделяющая граница имеет ограничения — состоит из гиперплоскостей, параллельных осям
- Необходимость борьбы с переобучением: стрижка или критерии останова
- Поиск оптимального дерева — NP-полная задача, поэтому на практике используется жадное построение

Неустойчивость отдельного дерева — не только недостаток: именно она делает деревья отличными базовыми моделями для ансамблей — случайного леса и градиентного бустинга.

Что мы обсудили

- 1 **kNN** — модель без обучения: ответ определяется k ближайшими объектами выборки
- 2 **Метрика и масштаб** — расстояние задаёт весь алгоритм, поэтому признаки обязательно масштабируют
- 3 **Наивный Байес** — теорема Байеса плюс допущение о независимости признаков внутри класса
- 4 **Решающее дерево** — бинарное дерево предикатов, строится жадно по функционалу $Q(R, j, t)$
- 5 **Критерии информативности** — дисперсия в регрессии, Джини и энтропия в классификации
- 6 **Сложность дерева** — ограничивается критериями останова или стрижкой готового дерева
- 7 **Пропуски и категории** — дерево обрабатывает их напрямую, без предварительной подготовки данных